

EmbeddedSystem_Robot_Buggy_Controller

Generated by Doxygen 1.15.0

1 Main Page	1
1.1 dsPIC33 Buggy Control Firmware	1
1.1.1 System Architecture	1
1.1.2 Finite State Machine (FSM)	1
1.1.2.1 1. STATE_HALTED	1
1.1.2.2 2. STATE_MOVING	1
1.1.2.3 3. STATE_OBSTACLE_AVOIDANCE	2
1.1.3 Real-Time Task Scheduler Matrix	2
1.1.4 H-Bridge Actuation Logic Mapping	3
1.1.5 The bluetooth interface (Linux/macOS Native)	3
1.1.5.1 Native Execution (Linux/macOS)	3
1.1.6 Project Documentation	3
2 Data Structure Index	5
2.1 Data Structures	5
3 File Index	7
3.1 File List	7
4 Data Structure Documentation	9
4.1 adcState Struct Reference	9
4.1.1 Detailed Description	9
4.1.2 Field Documentation	9
4.1.2.1 battery_voltage	9
4.1.2.2 ir_distance	9
4.2 ButtonState Struct Reference	9
4.2.1 Detailed Description	9
4.2.2 Field Documentation	10
4.2.2.1 debounce_counter	10
4.2.2.2 prev_state	10
4.3 circular_buffer Struct Reference	10
4.3.1 Detailed Description	10
4.3.2 Field Documentation	10
4.3.2.1 data	10
4.3.2.2 idxH	10
4.3.2.3 idxT	10
4.3.2.4 size	10
4.4 IMUdata Struct Reference	10
4.4.1 Detailed Description	11
4.4.2 Field Documentation	11
4.4.2.1 x	11
4.4.2.2 y	11
4.4.2.3 z	11

4.5 MissionControl_t Struct Reference	11
4.5.1 Detailed Description	11
4.5.2 Field Documentation	11
4.5.2.1 current_phase	11
4.5.2.2 next_phase	11
4.5.2.3 obstacle_counter	12
4.5.2.4 phase_counter	12
4.6 RobotState_t Struct Reference	12
4.6.1 Detailed Description	12
4.6.2 Field Documentation	12
4.6.2.1 attitude	12
4.6.2.2 cmd_vel	12
4.6.2.3 io	12
4.6.2.4 mission	12
4.6.2.5 sensors	12
4.7 SystemAttitude_t Struct Reference	13
4.7.1 Detailed Description	13
4.7.2 Field Documentation	13
4.7.2.1 pitch	13
4.7.2.2 relative_yaw	13
4.7.2.3 roll	13
4.7.2.4 yaw	13
4.8 SystemIO_t Struct Reference	13
4.8.1 Detailed Description	13
4.8.2 Field Documentation	14
4.8.2.1 btn_re8	14
4.8.2.2 btn_re9	14
4.8.2.3 char_received	14
4.8.2.4 rx_cmd	14
4.8.2.5 tx_buffer	14
4.9 SystemSensors_t Struct Reference	14
4.9.1 Detailed Description	14
4.9.2 Field Documentation	14
4.9.2.1 acc_raw	14
4.9.2.2 acc_x	15
4.9.2.3 acc_y	15
4.9.2.4 acc_z	15
4.9.2.5 adc_status	15
4.9.2.6 battery_voltage	15
4.9.2.7 gyr_raw	15
4.9.2.8 ir_distance	15
4.9.2.9 mag_raw	15

4.10 tasky Struct Reference	15
4.10.1 Detailed Description	15
4.10.2 Field Documentation	16
4.10.2.1 arg	16
4.10.2.2 enable	16
4.10.2.3 f	16
4.10.2.4 N	16
4.10.2.5 n	16
4.11 Twist_t Struct Reference	16
4.11.1 Detailed Description	16
4.11.2 Field Documentation	16
4.11.2.1 v	16
4.11.2.2 w	16
5 File Documentation	17
5.1 include/core/config.h File Reference	17
5.1.1 Detailed Description	18
5.1.2 Macro Definition Documentation	18
5.1.2.1 ACC_CHIP_ID	18
5.1.2.2 ACC_CS_LAT	18
5.1.2.3 ACC_CS_TRIS	18
5.1.2.4 BAUDRATE	18
5.1.2.5 BRGVAL	18
5.1.2.6 BTN_T1_PORT	19
5.1.2.7 BTN_T1_TRIS	19
5.1.2.8 BTN_T2_PORT	19
5.1.2.9 BTN_T2_TRIS	19
5.1.2.10 FCY	19
5.1.2.11 GYR_CHIP_ID	19
5.1.2.12 GYR_CS_LAT	19
5.1.2.13 GYR_CS_TRIS	19
5.1.2.14 LED1_LAT	19
5.1.2.15 LED1_TRIS	19
5.1.2.16 LIL_LAT	19
5.1.2.17 LIL_TRIS	19
5.1.2.18 LSL_LAT	20
5.1.2.19 LSL_TRIS	20
5.1.2.20 MAG_CHIP_ID	20
5.1.2.21 MAG_CS_LAT	20
5.1.2.22 MAG_CS_TRIS	20
5.1.2.23 MAG_OFFSET_X	20
5.1.2.24 MAG_OFFSET_Y	20

5.1.2.25 MAG_OFFSET_Z	20
5.1.2.26 MSG_ERR_INPUT	20
5.1.2.27 RAD_TO_DEG	20
5.1.2.28 RSL_LAT	20
5.1.2.29 RSL_TRIS	20
5.1.2.30 SPI1_MISO_RPIN	21
5.1.2.31 SPI1_MOSI_RPIN	21
5.1.2.32 SPI1_PPRE	21
5.1.2.33 SPI1_SCK_RPIN	21
5.1.2.34 SPI1_SPRE	21
5.1.2.35 UART1_RX_RPIN	21
5.1.2.36 UART1_TX_RPIN	21
5.2 config.h	21
5.3 include/core/fsm.h File Reference	22
5.3.1 Detailed Description	23
5.3.2 Enumeration Type Documentation	23
5.3.2.1 Event_t	23
5.3.2.2 State_t	23
5.3.3 Function Documentation	23
5.3.3.1 dispatch_event()	23
5.3.3.2 fsm_init()	24
5.3.3.3 get_current_state()	24
5.4 fsm.h	24
5.5 include/core/robot_state.h File Reference	25
5.5.1 Detailed Description	25
5.5.2 Enumeration Type Documentation	26
5.5.2.1 ObAvPhase_t	26
5.6 robot_state.h	26
5.7 include/core/scheduler.h File Reference	27
5.7.1 Detailed Description	27
5.7.2 Macro Definition Documentation	27
5.7.2.1 tick_ms	27
5.7.3 Function Documentation	28
5.7.3.1 disable_task()	28
5.7.3.2 enable_task()	28
5.7.3.3 register_task()	28
5.7.3.4 scheduler()	30
5.8 scheduler.h	30
5.9 include/core/tasks.h File Reference	31
5.9.1 Detailed Description	31
5.9.2 Function Documentation	31
5.9.2.1 hw_init()	31

5.9.2.2 tasks_init()	32
5.10 tasks.h	33
5.11 include/drivers/adc.h File Reference	33
5.11.1 Detailed Description	33
5.11.2 Function Documentation	34
5.11.2.1 adc_converter()	34
5.11.2.2 adc_init()	34
5.11.2.3 adc_scan_auto()	34
5.12 adc.h	34
5.13 include/drivers/BOSCH_BMX055.h File Reference	35
5.13.1 Detailed Description	35
5.13.2 Function Documentation	36
5.13.2.1 BMX055_mag_wakeup()	36
5.13.2.2 BMX055_setup()	36
5.13.2.3 imuRead()	36
5.14 BOSCH_BMX055.h	37
5.15 include/drivers/button.h File Reference	37
5.15.1 Detailed Description	37
5.15.2 Macro Definition Documentation	37
5.15.2.1 MAXDEBCYCLE	37
5.15.3 Function Documentation	38
5.15.3.1 button_has_just_been_pressed()	38
5.15.3.2 button_init()	38
5.16 button.h	38
5.17 include/drivers/pwm.h File Reference	39
5.17.1 Detailed Description	39
5.17.2 Function Documentation	39
5.17.2.1 execute_motion()	39
5.17.2.2 pwm_init()	40
5.17.2.3 pwm_set_duty_cycle()	40
5.18 pwm.h	40
5.19 include/drivers/spi.h File Reference	40
5.19.1 Detailed Description	40
5.19.2 Function Documentation	41
5.19.2.1 spi1_mikrobus1_setup()	41
5.19.2.2 spi1_write()	41
5.20 spi.h	41
5.21 include/drivers/timer.h File Reference	42
5.21.1 Detailed Description	42
5.21.2 Function Documentation	42
5.21.2.1 tmr_setup_period()	42
5.21.2.2 tmr_wait_ms()	44

5.21.2.3 tmr_wait_period()	44
5.22 timer.h	46
5.23 include/drivers/uart.h File Reference	46
5.23.1 Detailed Description	47
5.23.2 Macro Definition Documentation	47
5.23.2.1 UART_RX_BUF_SIZE	47
5.23.2.2 UART_TX_BUF_SIZE	47
5.23.3 Function Documentation	47
5.23.3.1 _U1RXInterrupt()	47
5.23.3.2 _U1TXInterrupt()	47
5.23.3.3 uart1_get_rx_count()	47
5.23.3.4 uart1_get_tx_count()	48
5.23.3.5 uart1_mikrobus2_setup()	48
5.23.3.6 uart1_read_char()	48
5.23.3.7 uart1_write()	48
5.24 uart.h	50
5.25 src/core/fsm.c File Reference	50
5.25.1 Detailed Description	51
5.25.2 Function Documentation	51
5.25.2.1 dispatch_event()	51
5.25.2.2 evaluate_obstacle()	51
5.25.2.3 fsm_init()	52
5.25.2.4 get_current_state()	52
5.25.2.5 handle_halt()	52
5.25.2.6 handle_move()	52
5.25.2.7 handle_obstacle()	53
5.25.3 Variable Documentation	53
5.25.3.1 state_table	53
5.26 src/core/main.c File Reference	53
5.26.1 Detailed Description	54
5.26.2 Function Documentation	54
5.26.2.1 main()	54
5.27 src/core/scheduler.c File Reference	54
5.27.1 Detailed Description	54
5.27.2 Function Documentation	55
5.27.2.1 disable_task()	55
5.27.2.2 enable_task()	55
5.27.2.3 register_task()	55
5.27.2.4 scheduler()	55
5.28 src/core/tasks.c File Reference	55
5.28.1 Detailed Description	57
5.28.2 Function Documentation	57

5.28.2.1 blink_A0()	57
5.28.2.2 blink_lights()	57
5.28.2.3 button_polling()	57
5.28.2.4 car_stopped()	57
5.28.2.5 hw_init()	59
5.28.2.6 move()	59
5.28.2.7 read_acc_mag()	59
5.28.2.8 read_gyro()	59
5.28.2.9 read_IR()	59
5.28.2.10 tasks_init()	60
5.28.2.11 UART_bat_log()	60
5.28.2.12 UART_get_twist()	60
5.28.2.13 UART_log()	60
5.28.2.14 UART_log_bt()	60
5.29 src/drivers/adc.c File Reference	61
5.29.1 Detailed Description	61
5.29.2 Function Documentation	61
5.29.2.1 adc_converter()	61
5.29.2.2 adc_init()	61
5.29.2.3 adc_scan_auto()	61
5.30 src/drivers/BOSCH_BMX055.c File Reference	62
5.30.1 Detailed Description	62
5.30.2 Function Documentation	62
5.30.2.1 BMX055_mag_wakeup()	62
5.30.2.2 BMX055_setup()	63
5.30.2.3 format12bit()	63
5.30.2.4 format13bit()	63
5.30.2.5 format15bit()	63
5.30.2.6 format16bit()	64
5.30.2.7 imuRead()	64
5.31 src/drivers/button.c File Reference	64
5.31.1 Detailed Description	64
5.31.2 Function Documentation	65
5.31.2.1 button_has_just_been_pressed()	65
5.31.2.2 button_init()	65
5.32 src/drivers/pwm.c File Reference	65
5.32.1 Detailed Description	66
5.32.2 Function Documentation	66
5.32.2.1 clamp()	66
5.32.2.2 execute_motion()	66
5.32.2.3 pwm_init()	66
5.32.2.4 pwm_set_duty_cycle()	66

5.33 src/drivers/spi.c File Reference	67
5.33.1 Detailed Description	67
5.33.2 Function Documentation	67
5.33.2.1 spi1_mikrobus1_setup()	67
5.33.2.2 spi1_write()	67
5.34 src/drivers/timer.c File Reference	68
5.34.1 Detailed Description	68
5.34.2 Function Documentation	68
5.34.2.1 tmr_setup_period()	68
5.34.2.2 tmr_wait_ms()	68
5.34.2.3 tmr_wait_period()	70
5.35 src/drivers/uart.c File Reference	70
5.35.1 Detailed Description	71
5.35.2 Function Documentation	71
5.35.2.1 _U1RXInterrupt()	71
5.35.2.2 _U1TXInterrupt()	71
5.35.2.3 uart1_get_rx_count()	71
5.35.2.4 uart1_get_tx_count()	71
5.35.2.5 uart1_mikrobus2_setup()	72
5.35.2.6 uart1_read_char()	72
5.35.2.7 uart1_write()	72
Index	73

Chapter 1

Main Page

1.1 dsPIC33 Buggy Control Firmware

This repository contains the embedded control firmware for an autonomous buggy project. The architecture relies on a dual $O(1)$ deterministic Finite State Machine (FSM) running on a Microchip dsPIC33 microcontroller to provide predictable, real-time behavior without utilizing conditional switch-cases or blocking delays.

1.1.1 System Architecture

To guarantee real-time safety, the firmware completely isolates spontaneous state transitions from continuous fixed-frequency state actions.

- **The Transition Matrix ([state_table](#)):** A 2D function pointer array indexed directly by `[Current_State][Incoming_Event]` updates the system state instantaneously in $O(1)$ computing time upon event arrival.
 - **Cooperative Scheduler:** A time-sliced task engine driven by a 500 Hz TIMER1 heartbeat executes registered subsystem callback functions non-preemptively when their configured tick periods elapse.
-

1.1.2 Finite State Machine (FSM)

The system processes three primary functional operational states:

1.1.2.1 1. STATE_HALTED

- **Behavior:** The buggy initializes in this safe resting state with all four H-Bridge PWM duty cycles set strictly to 0%.
- **Indicators:** The left-side (RB8) and right-side (RF1) indicator lights flash synchronously at 1 Hz, while the low-intensity auxiliary lights (RG1) remain disabled.
- **Transitions:** Pressing the physical RE8 button shifts the machine context into [STATE_MOVING](#).

1.1.2.2 2. STATE_MOVING

- **Motor Actuation:** Wheels are driven dynamically at a 10 kHz PWM frequency. Duty cycles are calculated and updated inside a 500 Hz control loop based on UART velocity and yaw references.
 - **Indicators:** Both side indicator lights are turned OFF, while the low-intensity auxiliary lights are turned ON.
 - **Transitions:**
 - Pressing the RE8 button forces the system back to [STATE_HALTED](#).
-

- By default initial linear and angular velocities are both set to zero.
- To change the default speed the command `$PCREF,XX,YY*` where XX and YY are respectively linear and angular speed
- If the IR sensor polling loop detects an obstacle closer than the 35.0 cm safety threshold, the system dispatches an `EVENT_OBSTACLE_DETECTED` trigger, instantly pivoting to `STATE_OBSTACLE_AVOIDANCE`.

1.1.2.3 3. STATE_OBSTACLE_AVOIDANCE

- **Behavior Maneuver:** The buggy abandons forward commands to execute a sequential, non-blocking hardware recovery maneuver:
 1. **Stop:** The vehicle zeroes actuation and waits until accelerometer integration confirms total zero kinetic inertia.
 2. **Rotate CW:** Applies yaw actuation until the gyroscope tracks a relative heading beyond -80.0 degrees (empirically established).
 3. **Move Forward:** Advances forward continuously for exactly 2.0 seconds (1000 loop ticks) or stops instantly if another obstruction is detected.
 4. **Rotate CCW:** Applies counter-clockwise yaw actuation to realign the heading beyond +80.0 degrees.
- **Indicators:** The right-side lights blink at 1 Hz to signal maneuvering, the left lights stay dark, and low-intensity lights remain active.
- **Evaluation & Retries:** After concluding the directional sequence, the IR sensor is sampled again. If the path is verified clear (> 20.0 cm), the machine transitions back to `STATE_MOVING`. If blocked, the maneuver increments a tracker and repeats up to a maximum limit of 3 attempts before triggering an emergency shutdown to `STATE_HALTED`.

1.1.3 Real-Time Task Scheduler Matrix

Data collection and physical actions run concurrently at strict frequencies generated by the core 500 Hz hardware timer tick.

Subsystem Task	Loop Frequency	Description
IR Sensor Polling	500 Hz	Refreshes the ADC buffer for distance metrics, subtracts specific sensor hardware offsets, and monitors the immediate path for threshold breaches.
Movement Actuation	500 Hz	Evaluates active state requirements, processes avoidance sequences, and applies target output voltages to the H-Bridge.
Button Polling	100 Hz	Debounces and evaluates the RE8 input pin for state toggles, and checks RE9 to trigger a diagnostic UART buffer transmission (<code>\$MBUF</code>).
IMU Spatial Fusion	100 Hz	Interfaces via SPI/I2C to read the Accelerometer, Magnetometer, and Gyroscope registers, running trigonometric integrations to map roll, pitch, and relative yaw.
Bluetooth Telemetry	10 Hz	Formats and pushes NMEA spatial packages (<code>\$MDIST</code> , <code>\$MANGLE</code>) and high-level FSM diagnostics via the serial stream.
Command Parsing	Asynchronous	Unpacks and decodes asynchronous <code>\$PCREF, speed, yawrate*</code> frames from the serial queue while enforcing safety saturation limits.
Heartbeat & Logging	1 Hz	Alternates pin latches to display avoidance light codes, toggles the main-loop health LED (A0), and writes <code>\$MBAT</code> battery voltages over UART.

1.1.4 H-Bridge Actuation Logic Mapping

Directional momentum is provided by four independent Output Compare PWM generators running continuously at a fixed 10 kHz base frequency.

Direction Command	PWM Applied (Duty > 0)	Low-Level Ground Pin
Left Wheels Forward	RD2 / RP66	RD1 / RP65 = 0
Left Wheels Backward	RD1 / RP65	RD2 / RP66 = 0
Right Wheels Forward	RD4 / RP68	RD3 / RP67 = 0
Right Wheels Backward	RD3 / RP67	RD4 / RP68 = 0

1.1.5 The bluetooth interface (Linux/macOS Native)

This interface routes the system's UART telemetry data wirelessly over a **Bluetooth connection instead of a physical cable link**.

Because this interface relies on a shell script (`.sh`), it is natively compatible **only with POSIX operating systems like Linux and macOS**. Standard Windows environments (Command Prompt and PowerShell) cannot execute this file directly.

1.1.5.1 Native Execution (Linux/macOS)

In the root folder of the project, grant execution permissions and run:

```
chmod +x ./bluetooth_interface/bt_interface.sh
./bluetooth_interface/bt_interface.sh
```

1.1.6 Project Documentation

The source codebase is fully documented using professional DOXYGEN/JAVADOC-style comments.

- **HTML Interactive Portal:** You can view the offline interactive documentation website by opening `docs/html/index.html` in any web browser.
- **PDF Reference Manual:** A print-ready, comprehensive PDF version of the complete system documentation is available directly inside the repository at: `docs/latex/refman.pdf`

Chapter 2

Data Structure Index

2.1 Data Structures

Here are the data structures with brief descriptions:

adcState	Container for the processed telemetry metrics	9
ButtonState	Tracks the persistent state and debounce count of a specific hardware button	9
circular_buffer	Internal circular buffer structure for non-blocking data queuing	10
IMUdata	Container for a 3-axis spatial measurement	10
MissionControl_t	Tracks the logical execution progression of autonomous missions	11
RobotState_t	The root aggregation object holding the complete logical state of the robot	12
SystemAttitude_t	Groups all attitude and spatial orientation data	13
SystemIO_t	Manages Input/Output peripherals and communication buffers	13
SystemSensors_t	Groups all raw and filtered sensor reading telemetry	14
tasky	Internal structure to represent a scheduled task	15
Twist_t	Groups targeted velocity commands	16

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

include/core/ config.h	Central configuration and pin mapping definition header for the robot buggy	17
include/core/ fsm.h	Finite State Machine engine declaration header for the robot buggy platform	22
include/core/ robot_state.h	Central global data context tracking matrix for the robot buggy platform	25
include/core/ scheduler.h	Cooperative non-preemptive task scheduler header for the robot buggy	27
include/core/ tasks.h	Subsystem task abstractions and hardware setup declarations for the robot buggy	31
include/drivers/ adc.h	Header file for the Analog-to-Digital Converter driver	33
include/drivers/ BOSCH_BMX055.h	Header file for the Bosch BMX055 IMU SPI driver	35
include/drivers/ button.h	Header file for button initialization and debouncing logic	37
include/drivers/ pwm.h	Header file for Pulse Width Modulation (PWM) and motor control interface	39
include/drivers/ spi.h	Header file for the SPI interface driver and hardware configuration	40
include/drivers/ timer.h	Header file for hardware timer configuration and delay management routines	42
include/drivers/ uart.h	Header file for the non-blocking UART driver implementation using circular buffers	46
src/core/ fsm.c	Finite State Machine engine implementation for the robot buggy platform	50
src/core/ main.c	Application entry point and main scheduler loop for the buggy	53
src/core/ scheduler.c	Cooperative cyclic task scheduler implementation for the robot buggy platform	54
src/core/ tasks.c	Subsystem task implementations and peripheral hardware drivers for the buggy	55
src/drivers/ adc.c	Analog-to-Digital Converter (ADC) configuration and sampling routines	61
src/drivers/ BOSCH_BMX055.c	SPI driver implementation for the Bosch BMX055 IMU	62
src/drivers/ button.c	Implementation of button initialization and debouncing logic	64
src/drivers/ pwm.c	Pulse Width Modulation (PWM) and motor control interface	65

src/drivers/ spi.c	
SPI interface driver and hardware configuration	67
src/drivers/ timer.c	
Hardware timer configuration and delay management routines	68
src/drivers/ uart.c	
Non-blocking UART driver implementation using circular buffers and interrupts	70

Chapter 4

Data Structure Documentation

4.1 adcState Struct Reference

Container for the processed telemetry metrics.

```
#include <adc.h>
```

Data Fields

- float [battery_voltage](#)
- float [ir_distance](#)

4.1.1 Detailed Description

Container for the processed telemetry metrics.

4.1.2 Field Documentation

4.1.2.1 battery_voltage

```
float battery_voltage
```

Scaled battery voltage reading.

4.1.2.2 ir_distance

```
float ir_distance
```

Linearized distance from the IR sensor in cm.

The documentation for this struct was generated from the following file:

- include/drivers/[adc.h](#)

4.2 ButtonState Struct Reference

Tracks the persistent state and debounce count of a specific hardware button.

```
#include <button.h>
```

Data Fields

- int [prev_state](#)
- int [debounce_counter](#)

4.2.1 Detailed Description

Tracks the persistent state and debounce count of a specific hardware button.

4.2.2 Field Documentation

4.2.2.1 debounce_counter

```
int debounce_counter
```

Tick down counter to validate stable pin reads.

4.2.2.2 prev_state

```
int prev_state
```

The last verified stable state of the button.

The documentation for this struct was generated from the following file:

- include/drivers/[button.h](#)

4.3 circular_buffer Struct Reference

Internal circular buffer structure for non-blocking data queuing.

Data Fields

- volatile char * [data](#)
- volatile int [idxH](#)
- volatile int [idxT](#)
- volatile int [size](#)

4.3.1 Detailed Description

Internal circular buffer structure for non-blocking data queuing.

4.3.2 Field Documentation

4.3.2.1 data

```
volatile char* data
```

Pointer to the data array

4.3.2.2 idxH

```
volatile int idxH
```

The index we WRITE to

4.3.2.3 idxT

```
volatile int idxT
```

The index we READ from

4.3.2.4 size

```
volatile int size
```

Total capacity of the buffer

The documentation for this struct was generated from the following file:

- src/drivers/[uart.c](#)

4.4 IMUdata Struct Reference

Container for a 3-axis spatial measurement.

```
#include <BOSCH_BMX055.h>
```

Data Fields

- [int x](#)
- [int y](#)
- [int z](#)

4.4.1 Detailed Description

Container for a 3-axis spatial measurement.

4.4.2 Field Documentation

4.4.2.1 x

`int x`

Formatted measurement for the X-axis.

4.4.2.2 y

`int y`

Formatted measurement for the Y-axis.

4.4.2.3 z

`int z`

Formatted measurement for the Z-axis.

The documentation for this struct was generated from the following file:

- `include/drivers/BOSCH_BMX055.h`

4.5 MissionControl_t Struct Reference

Tracks the logical execution progression of autonomous missions.

```
#include <robot_state.h>
```

Data Fields

- [ObAvPhase_t current_phase](#)
- [ObAvPhase_t next_phase](#)
- `uint32_t` [phase_counter](#)
- `uint32_t` [obstacle_counter](#)

4.5.1 Detailed Description

Tracks the logical execution progression of autonomous missions.

4.5.2 Field Documentation

4.5.2.1 current_phase

[ObAvPhase_t](#) `current_phase`

Currently executing sub-phase of the avoidance maneuver.

4.5.2.2 next_phase

[ObAvPhase_t](#) `next_phase`

Scheduled subsequent phase upon completion.

4.5.2.3 obstacle_counter

`uint32_t obstacle_counter`

Tally of consecutive obstacles encountered.

4.5.2.4 phase_counter

`uint32_t phase_counter`

Internal clock tick counter used for timing specific phases.

The documentation for this struct was generated from the following file:

- [include/core/robot_state.h](#)

4.6 RobotState_t Struct Reference

The root aggregation object holding the complete logical state of the robot.

```
#include <robot_state.h>
```

Data Fields

- [SystemSensors_t](#) sensors
- [SystemAttitude_t](#) attitude
- [Twist_t](#) cmd_vel
- [MissionControl_t](#) mission
- [SystemIO_t](#) io

4.6.1 Detailed Description

The root aggregation object holding the complete logical state of the robot.

4.6.2 Field Documentation

4.6.2.1 attitude

[SystemAttitude_t](#) attitude

Spatial orientation block.

4.6.2.2 cmd_vel

[Twist_t](#) cmd_vel

Locomotion target tracking block.

4.6.2.3 io

[SystemIO_t](#) io

Peripheral and interaction tracking block.

4.6.2.4 mission

[MissionControl_t](#) mission

Autonomous logic state block.

4.6.2.5 sensors

[SystemSensors_t](#) sensors

System sensing and telemetry block.

The documentation for this struct was generated from the following file:

- [include/core/robot_state.h](#)

4.7 SystemAttitude_t Struct Reference

Groups all attitude and spatial orientation data.

```
#include <robot_state.h>
```

Data Fields

- float [roll](#)
- float [pitch](#)
- float [yaw](#)
- float [relative_yaw](#)

4.7.1 Detailed Description

Groups all attitude and spatial orientation data.

4.7.2 Field Documentation

4.7.2.1 pitch

```
float pitch
```

Computed Pitch angle (degrees).

4.7.2.2 relative_yaw

```
float relative_yaw
```

Integrated relative Yaw angle from gyroscope (degrees).

4.7.2.3 roll

```
float roll
```

Computed Roll angle (degrees).

4.7.2.4 yaw

```
float yaw
```

Computed absolute Yaw angle from magnetometer (degrees).

The documentation for this struct was generated from the following file:

- include/core/[robot_state.h](#)

4.8 SystemIO_t Struct Reference

Manages Input/Output peripherals and communication buffers.

```
#include <robot_state.h>
```

Data Fields

- [ButtonState](#) [btn_re8](#)
- [ButtonState](#) [btn_re9](#)
- int [char_received](#)
- char [tx_buffer](#) [64]
- char [rx_cmd](#) [16]

4.8.1 Detailed Description

Manages Input/Output peripherals and communication buffers.

4.8.2 Field Documentation

4.8.2.1 btn_re8

[ButtonState](#) btn_re8

Debouncing structure for Button T1.

4.8.2.2 btn_re9

[ButtonState](#) btn_re9

Debouncing structure for Button T2.

4.8.2.3 char_received

int char_received

Flag indicating active UART reception.

4.8.2.4 rx_cmd

char rx_cmd[16]

Dedicated reception parsing buffer.

4.8.2.5 tx_buffer

char tx_buffer[64]

Dedicated transmission text formatting buffer.

The documentation for this struct was generated from the following file:

- [include/core/robot_state.h](#)

4.9 SystemSensors_t Struct Reference

Groups all raw and filtered sensor reading telemetry.

```
#include <robot_state.h>
```

Data Fields

- [IMUdata](#) acc_raw
- [IMUdata](#) mag_raw
- [IMUdata](#) gyr_raw
- float [acc_x](#)
- float [acc_y](#)
- float [acc_z](#)
- float [battery_voltage](#)
- float [ir_distance](#)
- [adcState](#) adc_status

4.9.1 Detailed Description

Groups all raw and filtered sensor reading telemetry.

4.9.2 Field Documentation

4.9.2.1 acc_raw

[IMUdata](#) acc_raw

Raw XYZ vectors from the accelerometer.

4.9.2.2 acc_x

`float acc_x`

Filtered/Scaled Acceleration X (g).

4.9.2.3 acc_y

`float acc_y`

Filtered/Scaled Acceleration Y (g).

4.9.2.4 acc_z

`float acc_z`

Filtered/Scaled Acceleration Z (g).

4.9.2.5 adc_status

`adcState adc_status`

Internal structure holding ADC hardware states.

4.9.2.6 battery_voltage

`float battery_voltage`

Computed system battery level (Volts).

4.9.2.7 gyr_raw

`IMUdata gyr_raw`

Raw XYZ vectors from the gyroscope.

4.9.2.8 ir_distance

`float ir_distance`

Computed forward obstacle distance (cm).

4.9.2.9 mag_raw

`IMUdata mag_raw`

Raw XYZ vectors from the magnetometer.

The documentation for this struct was generated from the following file:

- `include/core/robot_state.h`

4.10 tasky Struct Reference

Internal structure to represent a scheduled task.

Data Fields

- `int n`
- `int N`
- `void(* f)(void *)`
- `void * arg`
- `int enable`

4.10.1 Detailed Description

Internal structure to represent a scheduled task.

4.10.2 Field Documentation

4.10.2.1 `arg`

`void* arg`

Argument to pass to the task function.

4.10.2.2 `enable`

`int enable`

Enable flag: 1 if active, 0 if disabled.

4.10.2.3 `f`

`void(* f) (void *)`

Function pointer to the task payload.

4.10.2.4 `N`

`int N`

Tick period for the task execution.

4.10.2.5 `n`

`int n`

Current tick counter for the task.

The documentation for this struct was generated from the following file:

- [src/core/scheduler.c](#)

4.11 `Twist_t` Struct Reference

Groups targeted velocity commands.

`#include <robot_state.h>`

Data Fields

- `int32_t v`
- `int32_t w`

4.11.1 Detailed Description

Groups targeted velocity commands.

4.11.2 Field Documentation

4.11.2.1 `v`

`int32_t v`

Target linear velocity percentage (-100 to 100).

4.11.2.2 `w`

`int32_t w`

Target angular yaw rate percentage (-100 to 100).

The documentation for this struct was generated from the following file:

- [include/core/robot_state.h](#)

Chapter 5

File Documentation

5.1 include/core/config.h File Reference

Central configuration and pin mapping definition header for the robot buggy.

```
#include <xc.h>
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <stdbool.h>
```

Macros

System Constants

- #define `FCY` 72000000UL
- #define `RAD_TO_DEG` 57.29577951f

UART Configuration

- #define `BAUDRATE` 9600
- #define `BRGVAL` ((`FCY` / `BAUDRATE`) / 4) - 1
- #define `UART1_RX_RPIN` 75
- #define `UART1_TX_RPIN` 64

SPI1 Configuration

- #define `SPI1_PPRE` 0b11
- #define `SPI1_SPRE` 0b000
- #define `SPI1_MISO_RPIN` 17
- #define `SPI1_MOSI_RPIN` 109
- #define `SPI1_SCK_RPIN` 108

Sensor Chip Selects (BOSCH BMX055 IMU)

- #define `ACC_CS_LAT` LATBbits.LATB3
- #define `ACC_CS_TRIS` TRISBbits.TRISB3
- #define `ACC_CHIP_ID` 0xFA
- #define `GYR_CS_LAT` LATBbits.LATB4
- #define `GYR_CS_TRIS` TRISBbits.TRISB4
- #define `GYR_CHIP_ID` 0x0F
- #define `MAG_CS_LAT` LATDbits.LATD6
- #define `MAG_CS_TRIS` TRISDbits.TRISD6
- #define `MAG_CHIP_ID` 0x32

Magnetometer Calibration Offsets

- `#define MAG_OFFSET_X 0.0f`
- `#define MAG_OFFSET_Y 0.0f`
- `#define MAG_OFFSET_Z 0.0f`

GPIO Mappings: LEDs, Buttons, and Lights

- `#define LED1_TRIS TRISAbits.TRISA0`
- `#define LED1_LAT LATAbits.LATA0`
- `#define BTN_T1_TRIS TRISEbits.TRISE8`
- `#define BTN_T1_PORT PORTEbits.RE8`
- `#define BTN_T2_TRIS TRISEbits.TRISE9`
- `#define BTN_T2_PORT PORTEbits.RE9`
- `#define LIL_TRIS TRISGbits.TRISG1`
- `#define LIL_LAT LATGbits.LATG1`
- `#define LSL_TRIS TRISBbits.TRISB8`
- `#define LSL_LAT LATBbits.LATB8`
- `#define RSL_TRIS TRISFbits.TRISF1`
- `#define RSL_LAT LATFbits.LATF1`

System Error Messages

- `#define MSG_ERR_INPUT "$ERR,1*\n"`

5.1.1 Detailed Description

Central configuration and pin mapping definition header for the robot buggy.

Author

Simone Canella, Samuele Isaia, Riccardo Parotelli

Version

1.0.0

This file centralizes all global constants, peripheral configuration parameters, software settings, and Peripheral Pin Select (PPS) hardware mappings for the dsPIC microcontroller. It ensures consistency across all compilation modules.

5.1.2 Macro Definition Documentation

5.1.2.1 ACC_CHIP_ID

```
#define ACC_CHIP_ID 0xFA
```

Accelerometer hardware ID

5.1.2.2 ACC_CS_LAT

```
#define ACC_CS_LAT LATBbits.LATB3
```

Accelerometer Chip Select Latch Register

5.1.2.3 ACC_CS_TRIS

```
#define ACC_CS_TRIS TRISBbits.TRISB3
```

Accelerometer Chip Select Direction Register

5.1.2.4 BAUDRATE

```
#define BAUDRATE 9600
```

Centralized Baudrate for PC data logging/command interface.

5.1.2.5 BRGVAL

```
#define BRGVAL ((FCY / BAUDRATE) / 4) - 1
```

Baud Rate Generator value calculated for High-Speed mode (BRGH = 1).

5.1.2.6 BTN_T1_PORT

```
#define BTN_T1_PORT PORTEbits.RE8  
Button T1 Port Reading Register
```

5.1.2.7 BTN_T1_TRIS

```
#define BTN_T1_TRIS TRISEbits.TRISE8  
Button T1 Direction Register
```

5.1.2.8 BTN_T2_PORT

```
#define BTN_T2_PORT PORTEbits.RE9  
Button T2 Port Reading Register
```

5.1.2.9 BTN_T2_TRIS

```
#define BTN_T2_TRIS TRISEbits.TRISE9  
Button T2 Direction Register
```

5.1.2.10 FCY

```
#define FCY 72000000UL  
Instruction Frequency (72 MHz).
```

5.1.2.11 GYR_CHIP_ID

```
#define GYR_CHIP_ID 0x0F  
Gyroscope hardware ID
```

5.1.2.12 GYR_CS_LAT

```
#define GYR_CS_LAT LATBbits.LATB4  
Gyroscope Chip Select Latch Register
```

5.1.2.13 GYR_CS_TRIS

```
#define GYR_CS_TRIS TRISBbits.TRISB4  
Gyroscope Chip Select Direction Register
```

5.1.2.14 LED1_LAT

```
#define LED1_LAT LATAbits.LATA0  
Status LED 1 Latch Register
```

5.1.2.15 LED1_TRIS

```
#define LED1_TRIS TRISAbits.TRISA0  
Status LED 1 Direction Register
```

5.1.2.16 LIL_LAT

```
#define LIL_LAT LATGbits.LATG1  
Low Intensity Lights Latch Register
```

5.1.2.17 LIL_TRIS

```
#define LIL_TRIS TRISGbits.TRISG1  
Low Intensity Lights Direction Register
```

5.1.2.18 LSL_LAT

```
#define LSL_LAT LATBbits.LATB8
```

Left Side Light Latch Register

5.1.2.19 LSL_TRIS

```
#define LSL_TRIS TRISBbits.TRISB8
```

Left Side Light Direction Register

5.1.2.20 MAG_CHIP_ID

```
#define MAG_CHIP_ID 0x32
```

Magnetometer hardware ID

5.1.2.21 MAG_CS_LAT

```
#define MAG_CS_LAT LATDbits.LATD6
```

Magnetometer Chip Select Latch Register

5.1.2.22 MAG_CS_TRIS

```
#define MAG_CS_TRIS TRISDbits.TRISD6
```

Magnetometer Chip Select Direction Register

5.1.2.23 MAG_OFFSET_X

```
#define MAG_OFFSET_X 0.0f
```

X-axis hard-iron calibration offset for the magnetometer.

5.1.2.24 MAG_OFFSET_Y

```
#define MAG_OFFSET_Y 0.0f
```

Y-axis hard-iron calibration offset for the magnetometer.

5.1.2.25 MAG_OFFSET_Z

```
#define MAG_OFFSET_Z 0.0f
```

Z-axis hard-iron calibration offset for the magnetometer.

5.1.2.26 MSG_ERR_INPUT

```
#define MSG_ERR_INPUT "$ERR,1*\n"
```

Standard serial error response for invalid commands.

5.1.2.27 RAD_TO_DEG

```
#define RAD_TO_DEG 57.29577951f
```

Conversion factor from Radians to Degrees ($180.0f / \pi$).

5.1.2.28 RSL_LAT

```
#define RSL_LAT LATFbits.LATF1
```

Right Side Light Latch Register

5.1.2.29 RSL_TRIS

```
#define RSL_TRIS TRISFbits.TRISF1
```

Right Side Light Direction Register

5.1.2.30 SPI1_MISO_RPIN

```
#define SPI1_MISO_RPIN 17
```

Remapped SPI1 MISO signal pin ID: RPI17 (RA1).

5.1.2.31 SPI1_MOSI_RPIN

```
#define SPI1_MOSI_RPIN 109
```

Remapped SPI1 MOSI signal pin ID: RP109 (RF13).

5.1.2.32 SPI1_PPRE

```
#define SPI1_PPRE 0b11
```

8:1 primary prescaler configuration bits.

5.1.2.33 SPI1_SCK_RPIN

```
#define SPI1_SCK_RPIN 108
```

Remapped SPI1 SCK signal pin ID: RP108 (RF12).

5.1.2.34 SPI1_SPRE

```
#define SPI1_SPRE 0b000
```

1:1 secondary prescaler configuration bits.

5.1.2.35 UART1_RX_RPIN

```
#define UART1_RX_RPIN 75
```

Remapped UART1 RX signal pin ID: RPI75 (RD11).

5.1.2.36 UART1_TX_RPIN

```
#define UART1_TX_RPIN 64
```

Remapped UART1 TX signal pin ID: RP64 (RD0).

5.2 config.h

[Go to the documentation of this file.](#)

```
00001
00011
00012 #ifndef CONFIG_H
00013 #define CONFIG_H
00014
00015 // --- Standard C Libraries ---
00016 #include <xc.h>
00017 #include <math.h>
00018 #include <stdio.h>
00019 #include <stdlib.h>
00020 #include <string.h>
00021 #include <stdint.h>
00022 #include <stdbool.h>
00023
00028 #define FCY 72000000UL
00029 #define RAD_TO_DEG 57.29577951f
00031
00036 #define BAUDRATE 9600
00037 #define BRGVAL ((FCY / BAUDRATE) / 4) - 1
00038
00039 // UART1 PPS (Peripheral Pin Select)
00040 #define UART1_RX_RPIN 75
00041 #define UART1_TX_RPIN 64
00043
00048 #define SPI1_PPRE 0b11
00049 #define SPI1_SPRE 0b000
00050
00051 // SPI1 PPS (Peripheral Pin Select)
00052 #define SPI1_MISO_RPIN 17
00053 #define SPI1_MOSI_RPIN 109
00054 #define SPI1_SCK_RPIN 108
```

```

00056
00061 // Accelerometer (RB3)
00062 #define ACC_CS_LAT LATBbits.LATB3
00063 #define ACC_CS_TRIS TRISBbits.TRISB3
00064 #define ACC_CHIP_ID 0xFA
00065
00066 // Gyroscope (RB4)
00067 #define GYR_CS_LAT LATBbits.LATB4
00068 #define GYR_CS_TRIS TRISBbits.TRISB4
00069 #define GYR_CHIP_ID 0x0F
00070
00071 // Magnetometer (RD6)
00072 #define MAG_CS_LAT LATDbits.LATD6
00073 #define MAG_CS_TRIS TRISDbits.TRISD6
00074 #define MAG_CHIP_ID 0x32
00076
00081 #define MAG_OFFSET_X 0.0f
00082 #define MAG_OFFSET_Y 0.0f
00083 #define MAG_OFFSET_Z 0.0f
00085
00090 // LED 1 (RA0)
00091 #define LED1_TRIS TRISAbits.TRISA0
00092 #define LED1_LAT LATAbits.LATA0
00093
00094 // Button T1 (RE8)
00095 #define BTN_T1_TRIS TRISEbits.TRISE8
00096 #define BTN_T1_PORT PORTEbits.RE8
00097
00098 // Button T2 (RE9)
00099 #define BTN_T2_TRIS TRISEbits.TRISE9
00100 #define BTN_T2_PORT PORTEbits.RE9
00101
00102 // Low-intensity lights (RG1)
00103 #define LIL_TRIS TRISGbits.TRISG1
00104 #define LIL_LAT LATGbits.LATG1
00105
00106 // Left-side light (RB8)
00107 #define LSL_TRIS TRISBbits.TRISB8
00108 #define LSL_LAT LATBbits.LATB8
00109
00110 // Right-side light (RF1)
00111 #define RSL_TRIS TRISFbits.TRISF1
00112 #define RSL_LAT LATFbits.LATF1
00114
00119 #define MSG_ERR_INPUT "$ERR,1*\n"
00121
00122 #endif // CONFIG_H

```

5.3 include/core/fsm.h File Reference

Finite State Machine engine declaration header for the robot buggy platform.

```

#include "config.h"
#include "robot_state.h"

```

Enumerations

- enum [State_t](#) { [STATE_HALTED](#) , [STATE_MOVING](#) , [STATE_OBSTACLE_AVOIDANCE](#) , [MAX_STATES](#) }
Enumeration of available master states in the robot's FSM.
- enum [Event_t](#) { [EVENT_RE8_PRESSED](#) , [EVENT_OBSTACLE_DETECTED](#) , [EVENT_MANEUVER_COMPLETE](#) , [MAX_EVENTS](#) }
Enumeration of internal system events capable of triggering a state transition.

Functions

- void [fsm_init](#) ([RobotState_t](#) *rover)
Initializes the FSM engine, placing it into the default halted state.
- void [dispatch_event](#) ([RobotState_t](#) *rover, [Event_t](#) event, void *payload)
Submits a system event to the FSM for evaluation and potential state transition.
- [State_t](#) [get_current_state](#) (void)
Retrieves the current operating state of the finite state machine.

5.3.1 Detailed Description

Finite State Machine engine declaration header for the robot buggy platform.

Author

Simone Canella, Samuele Isaia, Riccardo Parotelli

Version

1.0.0

5.3.2 Enumeration Type Documentation

5.3.2.1 Event_t

enum [Event_t](#)

Enumeration of internal system events capable of triggering a state transition.

Enumerator

EVENT_RE8_PRESSED	Physical toggle button was pressed.
EVENT_OBSTACLE_DETECTED	IR sensor distance threshold was breached.
EVENT_MANEUVER_COMPLETE	Autonomous evasion sequence finished.
MAX_EVENTS	Total number of events.

5.3.2.2 State_t

enum [State_t](#)

Enumeration of available master states in the robot's FSM.

Enumerator

STATE_HALTED	Robot is completely stopped.
STATE_MOVING	Robot is executing motion commands.
STATE_OBSTACLE_AVOIDANCE	Robot is executing an autonomous avoidance maneuver.
MAX_STATES	Total number of states.

5.3.3 Function Documentation

5.3.3.1 dispatch_event()

```
void dispatch_event (  
    RobotState\_t * rover,  
    Event\_t event,  
    void * payload)
```

Submits a system event to the FSM for evaluation and potential state transition.

Parameters

<i>rover</i>	Pointer to the shared global robot state structure.
<i>event</i>	The triggered event code.

<i>payload</i>	An optional pointer to contextual data related to the event.
----------------	--

Submits a system event to the FSM for evaluation and potential state transition.

Parameters

<i>rover</i>	Pointer to the robot state structure.
<i>event</i>	The event being dispatched.
<i>payload</i>	Optional payload associated with the event.

5.3.3.2 fsm_init()

```
void fsm_init (
    RobotState_t * rover)
```

Initializes the FSM engine, placing it into the default halted state.

Parameters

<i>rover</i>	Pointer to the shared global robot state structure.
--------------	---

Initializes the FSM engine, placing it into the default halted state.

Parameters

<i>rover</i>	Pointer to the robot state structure.
--------------	---------------------------------------

5.3.3.3 get_current_state()

```
State_t get_current_state (
    void )
```

Retrieves the current operating state of the finite state machine.

Returns

The current [State_t](#) value.

Retrieves the current operating state of the finite state machine.

Returns

The current state.

5.4 fsm.h

[Go to the documentation of this file.](#)

```
00001
00007
00008 #ifndef FSM_H
00009 #define FSM_H
00010
00011 #include "config.h"
00012 #include "robot_state.h"
00013
00017 typedef enum {
00018     STATE_HALTED,
00019     STATE_MOVING,
00020     STATE_OBSTACLE_AVOIDANCE,
00021     MAX_STATES
```

```

00022 } State_t;
00023
00027 typedef enum {
00028     EVENT_RE8_PRESSED,
00029     EVENT_OBSTACLE_DETECTED,
00030     EVENT_MANEUVER_COMPLETE,
00031     MAX_EVENTS
00032 } Event_t;
00033
00034 // --- Public Function Interfaces ---
00035
00040 void fsm_init(RobotState_t *rover);
00041
00048 void dispatch_event(RobotState_t *rover, Event_t event, void *payload);
00049
00054 State_t get_current_state(void);
00055
00056 #endif // FSM_H

```

5.5 include/core/robot_state.h File Reference

Central global data context tracking matrix for the robot buggy platform.

```

#include "config.h"
#include "adc.h"
#include "BOSCH_BMX055.h"
#include "button.h"

```

Data Structures

- struct [SystemSensors_t](#)
Groups all raw and filtered sensor reading telemetry.
- struct [SystemAttitude_t](#)
Groups all attitude and spatial orientation data.
- struct [Twist_t](#)
Groups targeted velocity commands.
- struct [MissionControl_t](#)
Tracks the logical execution progression of autonomous missions.
- struct [SystemIO_t](#)
Manages Input/Output peripherals and communication buffers.
- struct [RobotState_t](#)
The root aggregation object holding the complete logical state of the robot.

Enumerations

- enum [ObAvPhase_t](#) {
[OBAV_PHASE_STOP](#) = 0 , [OBAV_PHASE_ROTATE_CW](#) = 1 , [OBAV_PHASE_MOVE_FW](#) = 2 ,
[OBAV_PHASE_ROTATE_CCW](#) = 3 ,
[OBAV_PHASE_EXIT](#) = 4 }
Enumeration of sub-phases executed sequentially during obstacle avoidance.

5.5.1 Detailed Description

Central global data context tracking matrix for the robot buggy platform.

Author

Simone Canella, Samuele Isaia, Riccardo Parotelli

Version

1.0.0

5.5.2 Enumeration Type Documentation

5.5.2.1 ObAvPhase_t

enum [ObAvPhase_t](#)

Enumeration of sub-phases executed sequentially during obstacle avoidance.

Enumerator

OBAV_PHASE_STOP	Halt current movement and zero velocities.
OBAV_PHASE_ROTATE_CW	Perform initial 90-degree right turn.
OBAV_PHASE_MOVE_FW	Drive straight to bypass the obstacle.
OBAV_PHASE_ROTATE_CCW	Perform final 90-degree left turn to resume heading.
OBAV_PHASE_EXIT	Evaluate environment and exit maneuver.

5.6 robot_state.h

[Go to the documentation of this file.](#)

```

00001
00007
00008 #ifndef ROBOT_STATE_H
00009 #define ROBOT_STATE_H
00010
00011 #include "config.h"
00012 #include "adc.h"
00013 #include "BOSCH_BMX055.h"
00014 #include "button.h"
00015
00019 typedef enum {
00020     OBAV_PHASE_STOP           = 0,
00021     OBAV_PHASE_ROTATE_CW     = 1,
00022     OBAV_PHASE_MOVE_FW       = 2,
00023     OBAV_PHASE_ROTATE_CCW    = 3,
00024     OBAV_PHASE_EXIT          = 4
00025 } ObAvPhase_t;
00026
00027 // --- SUBSYSTEM DATA STRUCTURES ---
00028
00032 typedef struct {
00033     IMUdata acc_raw;
00034     IMUdata mag_raw;
00035     IMUdata gyr_raw;
00036     float acc_x;
00037     float acc_y;
00038     float acc_z;
00039     float battery_voltage;
00040     float ir_distance;
00041     adcState adc_status;
00042 } SystemSensors_t;
00043
00047 typedef struct {
00048     float roll;
00049     float pitch;
00050     float yaw;
00051     float relative_yaw;
00052 } SystemAttitude_t;
00053
00057 typedef struct {
00058     int32_t v;
00059     int32_t w;
00060 } Twist_t;
00061
00065 typedef struct {
00066     ObAvPhase_t current_phase;
00067     ObAvPhase_t next_phase;
00068     uint32_t phase_counter;
00069     uint32_t obstacle_counter;
00070 } MissionControl_t;
00071
00075 typedef struct {
00076     ButtonState btn_re8;
00077     ButtonState btn_re9;
00078     int char_received;
00079     char tx_buffer[64];
00080     char rx_cmd[16];
00081 } SystemIO_t;

```

```

00082
00083 // --- THE ROOT STATE STRUCT ---
00084 typedef struct {
00085     SystemSensors_t  sensors;
00086     SystemAttitude_t attitude;
00087     Twist_t          cmd_vel;
00088     MissionControl_t mission;
00089     SystemIO_t        io;
00090 } RobotState_t;
00091
00092 #endif // ROBOT_STATE_H

```

5.7 include/core/scheduler.h File Reference

Cooperative non-preemptive task scheduler header for the robot buggy.

```

#include "timer.h"
#include <stddef.h>

```

Macros

- `#define tick_ms 2`
Base scheduling time slice definition in milliseconds.

Functions

- void `scheduler` ()
Main task evaluation and execution loop handler.
- void `register_task` (int initial_offset, int period_ticks, void(*f)(void *), void *arg)
Registers a new periodic task into the background execution matrix.
- void `enable_task` (int task_ID)
Enables a registered task to resume its periodic execution loops.
- void `disable_task` (int task_ID)
Disables a registered task to temporarily halt its execution loop.

5.7.1 Detailed Description

Cooperative non-preemptive task scheduler header for the robot buggy.

Author

Simone Canella, Samuele Isaia, Riccardo Parotelli

Version

1.0.0

This module declares a lightweight cyclic scheduler operating on a fixed time base. It maintains a tracking matrix of registered system tasks and updates execution flags at every interval tick. Tasks execute completely to completion without nested thread interruption.

NOTE:

- The system tick interval is centralized here.
- All registered tasks must execute quickly to prevent timing overruns on subsequent tasks.

5.7.2 Macro Definition Documentation

5.7.2.1 tick_ms

```
#define tick_ms 2
```

Base scheduling time slice definition in milliseconds.

Defines the baseline physical clock period matching the hardware timer setup.

5.7.3 Function Documentation

5.7.3.1 `disable_task()`

```
void disable_task (
    int task_ID)
```

Disables a registered task to temporarily halt its execution loop.

Parameters

<i>task_ID</i>	The numeric identifier index of the target task.
----------------	--

Returns

void

Suspends a task from being triggered by clearing its active run flag. The task remains in memory with its scheduling context preserved, but it is skipped during the scheduler execution sweeps.

Disables a registered task to temporarily halt its execution loop.

Parameters

<i>task_ID</i>	The index of the task to disable.
----------------	-----------------------------------

5.7.3.2 `enable_task()`

```
void enable_task (
    int task_ID)
```

Enables a registered task to resume its periodic execution loops.

Parameters

<i>task_ID</i>	The numeric identifier index of the target task.
----------------	--

Returns

void

Modifies the control flag of the specified task index, authorizing the execution loop engine to evaluate its periodic counters again.

Enables a registered task to resume its periodic execution loops.

Parameters

<i>task_ID</i>	The index of the task to enable.
----------------	----------------------------------

5.7.3.3 `register_task()`

```
void register_task (
    int initial_offset,
    int period_ticks,
    void(* f )(void *),
    void * arg)
```

Registers a new periodic task into the background execution matrix.

Parameters

<i>initial_offset</i>	The starting delay in ticks before the first task execution occurs.
<i>period_ticks</i>	The repetition interval of the task measured in system clock ticks.
<i>f</i>	Function pointer targeting the specific task callback function.
<i>arg</i>	Generic void pointer containing optional data parameters passed to the task.

Returns

void

Adds a task structure into the internal registration array. The offset parameter is highly critical here: it is used to interleave task execution frames, ensuring that the processor worst-case scenario load is evenly balanced across separate ticks.

Registers a new periodic task into the background execution matrix.

Parameters

<i>initial_offset</i>	The initial offset in ticks before first execution.
<i>period_ticks</i>	The recurrence period in ticks.
<i>f</i>	Function pointer to the task routine.
<i>arg</i>	Pointer to pass to the task routine as an argument.

5.7.3.4 scheduler()

```
void scheduler (
    void )
```

Main task evaluation and execution loop handler.

Returns

void

This routine must be called inside the main infinite processing loop. It iterates through the matrix of registered tasks, decrements their remaining delay counts, and calls the function pointers of active tasks whose intervals have expired.

Main task evaluation and execution loop handler.

5.8 scheduler.h

[Go to the documentation of this file.](#)

```
00001
00016
00017 #ifndef SCHEDULER_H
00018 #define SCHEDULER_H
00019
00020 #include "timer.h"
00021 #include <stddef.h>
00022
00027 #define tick_ms 2          // tick duration in ms
00028
00029
00030 // --- Core Public Scheduler Interfaces ---
00031
00041 void scheduler();
00042
00055 void register_task(int initial_offset, int period_ticks, void (*f)(void *), void *arg);
00056
00065 void enable_task(int task_ID);
00066
00076 void disable_task(int task_ID);
00077
00078 #endif // SCHEDULER_H
```

5.9 include/core/tasks.h File Reference

Subsystem task abstractions and hardware setup declarations for the robot buggy.

```
#include "config.h"
#include "robot_state.h"
```

Functions

- void [tasks_init](#) (void *param)
Registers all system runtime operational tasks into the scheduler matrix.
- void [hw_init](#) ([RobotState_t](#) *rover)
Configures low-level microcontroller hardware registers and peripherals.

5.9.1 Detailed Description

Subsystem task abstractions and hardware setup declarations for the robot buggy.

Author

Simone Canella, Samuele Isaia, Riccardo Parotelli

Version

1.0.0

This header file declares the initialization interfaces responsible for configuring low-level microcontroller peripherals and registering periodic execution tasks with the cooperative system scheduler.

5.9.2 Function Documentation

5.9.2.1 hw_init()

```
void hw_init (
    RobotState\_t * rover)
```

Configures low-level microcontroller hardware registers and peripherals.

Parameters

<i>rover</i>	Pointer to the robot state structure to configure.
--------------	--

Returns

void

Sets up all required physical hardware components on the dsPIC core. This handles the initialization of data direction registers (TRIS), latch configurations (LAT), Peripheral Pin Select (PPS) pin remapping for UART/SPI communication links, analog-to-digital converters (ADC) for infrared and battery sensing, and Output Compare modules for the 10 kHz motor PWM waves.

Configures low-level microcontroller hardware registers and peripherals.

Parameters

<i>rover</i>	Pointer to the robot state structure to configure.
--------------	--

5.9.2.2 tasks_init()

```
void tasks_init (  
    void * param)
```

Registers all system runtime operational tasks into the scheduler matrix.

Parameters

<i>param</i>	Generic initialization parameter container representing the robot state.
--------------	--

Returns

void

Connects individual software tasks (such as sensor reading, state machine dispatching, motor actuation, and telemetry logging) with the periodic cyclic scheduler. In order to guarantee maximum execution time budgets for every single clock tick, tasks are registered with pre-calculated initial offsets so that their worst-case workloads do not overlap.

Registers all system runtime operational tasks into the scheduler matrix.

Parameters

<i>param</i>	Pointer to the robot state structure to be passed into tasks.
--------------	---

5.10 tasks.h

[Go to the documentation of this file.](#)

```
00001
00011
00012 #ifndef TASKS_H
00013 #define TASKS_H
00014
00015 #include "config.h"
00016 #include "robot_state.h"
00017
00018 // --- Public Initialization Interfaces ---
00019
00030 void tasks_init(void *param);
00031
00042 void hw_init(RobotState_t *rover);
00043
00044 #endif // TASKS_H
```

5.11 include/drivers/adc.h File Reference

Header file for the Analog-to-Digital Converter driver.

```
#include "config.h"
```

Data Structures

- struct [adcState](#)

Container for the processed telemetry metrics.

Functions

- void [adc_init](#) (uint8_t channel)
Initializes the ADC hardware and configures target analog pins.
- double [adc_scan_auto](#) (uint8_t channel)
Executes a sample request with automatic hardware conversion.
- void [adc_converter](#) ([adcState](#) *adc)
Core processing task to sample sensors and compute physical telemetry metrics.

5.11.1 Detailed Description

Header file for the Analog-to-Digital Converter driver.

5.11.2 Function Documentation

5.11.2.1 `adc_converter()`

```
void adc_converter (
    adcState * adc)
```

Core processing task to sample sensors and compute physical telemetry metrics.

Parameters

<i>adc</i>	Pointer to the global state structure to store final converted metrics.
<i>adc</i>	Pointer to the <code>adcState</code> structure to populate with metrics.

5.11.2.2 `adc_init()`

```
void adc_init (
    uint8_t channel)
```

Initializes the ADC hardware and configures target analog pins.

Parameters

<i>channel</i>	The specific channel (11 or 15) to lock the multiplexer onto.
<i>channel</i>	The analog channel to initialize.

5.11.2.3 `adc_scan_auto()`

```
double adc_scan_auto (
    uint8_t channel)
```

Executes a sample request with automatic hardware conversion.

Parameters

<i>channel</i>	The specific channel (11 or 15) to lock the multiplexer onto.
----------------	---

Returns

float The raw ADC buffer value.

Parameters

<i>channel</i>	The analog channel to scan.
----------------	-----------------------------

Returns

double The raw ADC conversion result.

5.12 `adc.h`

[Go to the documentation of this file.](#)

```
00001
00005
00006 #ifndef ADC_H
```

```

00007 #define ADC_H
00008
00009 #include "config.h"
00010
00015 typedef struct{
00016     float battery_voltage;
00017     float ir_distance;
00018 } adcState;
00019
00024 void adc_init(uint8_t channel);
00025
00031 double adc_scan_auto(uint8_t channel);
00032
00037 void adc_converter(adcState *adc);
00038
00039 #endif // ADC_H

```

5.13 include/drivers/BOSCH_BMX055.h File Reference

Header file for the Bosch BMX055 IMU SPI driver.

```
#include "config.h"
```

Data Structures

- struct [IMUdata](#)
Container for a 3-axis spatial measurement.

Macros

- #define **ACC** 1
Target identifier for the Accelerometer.
- #define **GYR** 2
Target identifier for the Gyroscope.
- #define **MAG** 3
Target identifier for the Magnetometer.
- #define **ACC_CS** LATBbits.LATB3
Chip Select pin mapping for the Accelerometer.
- #define **GYR_CS** LATBbits.LATB4
Chip Select pin mapping for the Gyroscope.
- #define **MAG_CS** LATDbits.LATD6
Chip Select pin mapping for the Magnetometer.

Functions

- void [BMX055_setup](#) ()
Initializes the BMX055 sensor and configures the SPI bus parameters.
- void [BMX055_mag_wakeup](#) ()
Executes the wake-up sequence for the BMX055 magnetometer.
- [IMUdata imuRead](#) (int sensor)
Reads a full frame of multi-axis data from the specified IMU sensor.

5.13.1 Detailed Description

Header file for the Bosch BMX055 IMU SPI driver.

5.13.2 Function Documentation

5.13.2.1 BMX055_mag_wakeup()

```
void BMX055_mag_wakeup ()
```

Executes the wake-up sequence for the BMX055 magnetometer.

- Forces the magnetometer interface to switch from I2C to SPI by toggling the MAG_CS pin.
- Transitions the magnetometer from Suspend Mode to Sleep Mode by writing 0x01 to register 0x4B.
- Transitions the magnetometer from Sleep Mode to Active Mode by writing 0x00 to register 0x4C.
- Utilizes TIMER1 delays between transmissions to allow for power mode switching times.

5.13.2.2 BMX055_setup()

```
void BMX055_setup ()
```

Initializes the BMX055 sensor and configures the SPI bus parameters.

- Sets Chip Select (CS) lines for the Accelerometer, Gyroscope, and Magnetometer to high (inactive).
- Temporarily disables the SPI module to configure protocol settings matching the BMX055 requirements.
- Configures the SPI clock polarity to idle LOW (CKP = 0) and sets data changes to occur on transition from active to idle (CKE = 1).
- Sets the accelerometer's initial bandwidth to 1000 Hz by writing value 15 to register 0x10.

5.13.2.3 imuRead()

```
IMUdata imuRead (
    int sensor)
```

Reads a full frame of multi-axis data from the specified IMU sensor.

Parameters

<i>sensor</i>	The target sensor peripheral to read (ACC, GYR, or MAG).
---------------	--

Returns

[IMUdata](#) A structure containing the extracted x, y, and z axis measurements.

- Dispatches SPI reads depending on the requested sensor macro ([ACC](#), [GYR](#), [MAG](#)).
- Sets the MSB of the register address to 1 (0x80) to indicate a read operation.
- Accelerometer (ACC): Reads from register 0x02 and formats X, Y, and Z axes as 12-bit integers.
- Gyroscope (GYR): Reads from register 0x02 and formats X, Y, and Z axes as 16-bit integers.
- Magnetometer (MAG): Reads from register 0x42, formats X and Y axes as 13-bit integers, and the Z axis as a 15-bit integer.

Parameters

<i>sensor</i>	The target sensor peripheral to read (ACC , GYR , or MAG).
---------------	--

Returns

[IMUdata](#) A structure containing the extracted x, y, and z axis measurements.

5.14 BOSCH_BMX055.h

[Go to the documentation of this file.](#)

```

00001
00005
00006 #ifndef BOSCH_BMX055_H
00007 #define BOSCH_BMX055_H
00008
00009     #include "config.h"
00010
00012     #define ACC 1
00014     #define GYR 2
00016     #define MAG 3
00017
00019     #define ACC_CS LATBbits.LATB3 // RB3 accelerometer
00021     #define GYR_CS LATBbits.LATB4 // RB4 gyroscope
00023     #define MAG_CS LATDbits.LATD6 // RD6 magnetometer
00024
00029     typedef struct {
00030         int x;
00031         int y;
00032         int z;
00033     } IMUdata;
00034
00038     void BMX055_setup();
00039
00043     void BMX055_mag_wakeup();
00044
00050     IMUdata imuRead(int sensor);
00051
00052 #endif // BOSCH_BMX055_H

```

5.15 include/drivers/button.h File Reference

Header file for button initialization and debouncing logic.

Data Structures

- struct [ButtonState](#)
Tracks the persistent state and debounce count of a specific hardware button.

Macros

- #define [MAXDEBCYCLE](#) 1
The debounce delay threshold. Adjust based on your polling timer frequency.

Functions

- void [button_init](#) ([ButtonState](#) *btn)
Initializes a button to its default (unpressed) state.
- int [button_has_just_been_pressed](#) ([ButtonState](#) *btn, int current_pin_state)
Evaluates the button state and handles debouncing.

5.15.1 Detailed Description

Header file for button initialization and debouncing logic.

5.15.2 Macro Definition Documentation

5.15.2.1 MAXDEBCYCLE

```
#define MAXDEBCYCLE 1
```

The debounce delay threshold. Adjust based on your polling timer frequency.

•

5.15.3 Function Documentation

5.15.3.1 `button_has_just_been_pressed()`

```
int button_has_just_been_pressed (
    ButtonState * btn,
    int current_pin_state)
```

Evaluates the button state and handles debouncing.

Parameters

<i>btn</i>	Pointer to the active ButtonState structure tracking this pin.
<i>current_pin_state</i>	The current digital reading of the physical button pin (0 or 1).

Returns

int Returns 1 if a valid press is detected, 0 otherwise.

- Returns 1 ONLY on the exact cycle the button is validly pressed. Returns 0 otherwise. Decrements the debounce counter if a state change is detected, and updates the state only when the counter reaches 0.

•

Parameters

<i>btn</i>	Pointer to the active ButtonState structure tracking this pin.
<i>current_pin_state</i>	The current digital reading of the physical button pin (0 or 1).

Returns

int Returns 1 if a valid press is detected, 0 otherwise.

5.15.3.2 `button_init()`

```
void button_init (
    ButtonState * btn)
```

Initializes a button to its default (unpressed) state.

Parameters

<i>btn</i>	Pointer to the ButtonState structure to be initialized.
------------	---

- Sets the previous state to 1, assuming pull-up resistors are used, and resets the debounce counter to MAXDEBCYCLE.

•

Parameters

<i>btn</i>	Pointer to the ButtonState structure to be initialized.
------------	---

5.16 `button.h`

[Go to the documentation of this file.](#)

```
00001
00005
00006 #ifndef BUTTON_H
```

```

00007 #define BUTTON_H
00008
00012 #define MAXDEBCYCLE 1
00013
00018 typedef struct {
00019     int prev_state;
00020     int debounce_counter;
00021 } ButtonState;
00022
00027 void button_init(ButtonState *btn);
00028
00035 int button_has_just_been_pressed(ButtonState *btn, int current_pin_state);
00036
00037 #endif // BUTTON_H

```

5.17 include/drivers/pwm.h File Reference

Header file for Pulse Width Modulation (PWM) and motor control interface.

```
#include "config.h"
```

Functions

- void `pwm_init` (void)
Initializes the motor control pins and Output Compare peripherals.
- int `pwm_set_duty_cycle` (int percent, unsigned int reg)
Converts a percentage into a hardware register duty cycle value.
- void `execute_motion` (int v, int w)
Computes differential kinematics and updates H-Bridge duty cycles.

5.17.1 Detailed Description

Header file for Pulse Width Modulation (PWM) and motor control interface.

5.17.2 Function Documentation

5.17.2.1 `execute_motion()`

```

void execute_motion (
    int v,
    int w)

```

Computes differential kinematics and updates H-Bridge duty cycles.

Parameters

v	Target linear velocity (-100 to 100).
w	Target angular yaw rate (-100 to 100).

Converts a target forward velocity and angular yaw rate into independent left and right track speeds. It applies threshold clamps and routes the resulting magnitudes to the appropriate forward or backward OC module.

Parameters

v	Target linear velocity (-100 to 100).
w	Target angular yaw rate (-100 to 100).

5.17.2.2 pwm_init()

```
void pwm_init (
    void )
```

Initializes the motor control pins and Output Compare peripherals.

Configures the directional pins (RD1-RD4) as outputs and maps them to specific Re-mappable Pins (RP65-RP68). It initializes four Output Compare modules (OC1-OC4) in edge-aligned PWM mode with a base frequency of 10 kHz.

5.17.2.3 pwm_set_duty_cycle()

```
int pwm_set_duty_cycle (
    int percent,
    unsigned int reg)
```

Converts a percentage into a hardware register duty cycle value.

Parameters

<i>percent</i>	The desired duty cycle ranging from 0 to 100.
<i>reg</i>	The period register boundary (OCxRS) to scale against.

Returns

int The calculated hardware integer to be written to OCxR.

5.18 pwm.h

[Go to the documentation of this file.](#)

```
00001
00005
00006 #ifndef PWM_H
00007 #define PWM_H
00008
00009 #include "config.h"
00010
00014 void pwm_init(void);
00015
00022 int pwm_set_duty_cycle(int percent, unsigned int reg);
00023
00029 void execute_motion(int v, int w);
00030
00031 #endif // PWM_H
```

5.19 include/drivers/spi.h File Reference

Header file for the SPI interface driver and hardware configuration.

```
#include "config.h"
```

Functions

- void [spi1_mikrobus1_setup](#) ()
Configures the SPI1 peripheral pins, control registers, and default chip selects.
- unsigned int [spi1_write](#) (unsigned int data)
Transmits and receives a single byte over the SPI bus.

5.19.1 Detailed Description

Header file for the SPI interface driver and hardware configuration.

5.19.2 Function Documentation

5.19.2.1 spi1_mikrobus1_setup()

```
void spil_mikrobus1_setup ()
```

Configures the SPI1 peripheral pins, control registers, and default chip selects.

- Maps the MISO line to input pin RA1 (RP117).
- Maps the MOSI line to output pin RF13 (RP109).
- Maps the SCK (Serial Clock) line to output pin RF12 (RP108).
- Initializes three independent Chip Select (CS) pins (RB3, RB4, RD6) as digital outputs idling HIGH (inactive).
- Configures the SPI core for Master Mode, 8-bit data width, and sets prescalers to control the SCK frequency.
- Enables the SPI1 module.

5.19.2.2 spi1_write()

```
unsigned int spil_write (
    unsigned int data)
```

Transmits and receives a single byte over the SPI bus.

Parameters

<i>data</i>	The 8-bit data payload to transmit.
-------------	-------------------------------------

Returns

unsigned int The 8-bit data payload received from the slave device.

This is a blocking, synchronous hardware call. SPI communication is full-duplex, meaning a byte is always received simultaneously as a byte is transmitted.

Parameters

<i>data</i>	The 8-bit data payload to transmit.
-------------	-------------------------------------

Returns

unsigned int The 8-bit data payload received from the slave device.

5.20 spi.h

[Go to the documentation of this file.](#)

```
00001
00005
00006 #ifndef SPI_H
00007 #define SPI_H
00008
00009     #include "config.h"
00010
00014     void spil_mikrobus1_setup();
00015
00021     unsigned int spil_write(unsigned int data);
00022
00023 #endif // SPI_H
```

5.21 include/drivers/timer.h File Reference

Header file for hardware timer configuration and delay management routines.

```
#include <xc.h>
```

Macros

- **#define TIMER1 1**
Identifier for hardware TIMER1.
- **#define TIMER2 2**
Identifier for hardware TIMER2.
- **#define TIMER3 3**
Identifier for hardware TIMER3.
- **#define TIMER4 4**
Identifier for hardware TIMER4.
- **#define TIMER5 5**
Identifier for hardware TIMER5.
- **#define TMREXP 1**
Error code flag indicating the timer expired before waiting began.
- **#define NOTMR -2**
Error code flag indicating the specified timer is not valid.

Functions

- `int tmr_setup_period (int tmr, float ms)`
Configures and starts a specific timer for a given period.
- `int tmr_wait_period (int tmr)`
Performs a blocking wait until the specified timer period expires.
- `int tmr_wait_ms (int tmr, float ms)`
Executes a synchronous, one-shot blocking delay.

5.21.1 Detailed Description

Header file for hardware timer configuration and delay management routines.

5.21.2 Function Documentation

5.21.2.1 tmr_setup_period()

```
int tmr_setup_period (
    int tmr,
    float ms)
```

Configures and starts a specific timer for a given period.

Parameters

<i>tmr</i>	The hardware timer to configure (e.g., TIMER1, TIMER2, ..., TIMER5).
<i>ms</i>	The desired timer period in milliseconds.

Returns

int Returns 0 on successful configuration, or NOTMR if an invalid timer is passed.

- Dynamically selects the optimal prescaler (1:8, 1:64, or 1:256) based on the target milliseconds to avoid register overflow. It resets the timer, clears its interrupt flag, configures the period register (PRx), and starts it.

- **Parameters**

<i>tmr</i>	The hardware timer to configure (e.g., TIMER1, TIMER2, ..., TIMER5).
<i>ms</i>	The desired timer period in milliseconds. Max supported period is ~232 ms.

Returns

int Returns 0 on successful configuration, or NOTMR if an invalid timer is passed.

5.21.2.2 tmr_wait_ms()

```
int tmr_wait_ms (
    int tmr,
    float ms)
```

Executes a synchronous, one-shot blocking delay.

Parameters

<i>tmr</i>	The hardware timer to use for the delay (e.g., TIMER1, TIMER2, ..., TIMER5).
<i>ms</i>	The duration to block execution, in milliseconds.

Returns

int Returns 0 upon completion, or NOTMR if an invalid timer is passed.

- A convenience wrapper that automatically sets up the timer, waits for the requested period to expire, and cleanly shuts down the timer afterward to prevent continuous background counting.

-

Parameters

<i>tmr</i>	The hardware timer to use for the delay (e.g., TIMER1, TIMER2, ..., TIMER5).
<i>ms</i>	The duration to block execution, in milliseconds.

Returns

int Returns 0 upon completion, or NOTMR if an invalid timer is passed.

5.21.2.3 tmr_wait_period()

```
int tmr_wait_period (
    int tmr)
```

Performs a blocking wait until the specified timer period expires.

Parameters

<i>tmr</i>	The hardware timer to monitor (e.g., TIMER1, TIMER2, ..., TIMER5).
------------	--

Returns

int Returns 0 upon successful wait completion, TMREXP if deadline missed, or NOTMR on error.

- Polls the hardware interrupt flag for the given timer. If the flag is already set when this function is called, it detects a missed deadline. Otherwise, it holds execution in a while-loop until the hardware timer trips the flag.

- **Parameters**

<i>tmr</i>	The hardware timer to monitor (e.g., TIMER1, TIMER2, ..., TIMER5).
------------	--

Returns

int Returns 0 upon successful wait completion, TMREXP if the deadline was missed before polling began, or NOTMR for an invalid timer.

5.22 timer.h

[Go to the documentation of this file.](#)

```
00001
00005
00006 #ifndef TIMER_H_
00007 #define TIMER_H_
00008
00009 #include <xc.h>
00010
00012 #define TIMER1 1
00014 #define TIMER2 2
00016 #define TIMER3 3
00018 #define TIMER4 4
00020 #define TIMER5 5
00021
00023 #define TMREXP 1 // timer expired flag
00025 #define NOTMR -2 // timer is not defined
00026
00033 int tmr_setup_period(int tmr, float ms);
00034
00040 int tmr_wait_period(int tmr);
00041
00048 int tmr_wait_ms(int tmr, float ms);
00049
00050 #endif // TIMER_H_
```

5.23 include/drivers/uart.h File Reference

Header file for the non-blocking UART driver implementation using circular buffers.

```
#include "config.h"
```

Macros

- `#define UART_RX_BUF_SIZE 256`
Size allocation for the UART RX software ring buffer. Length of single correct command: 6 bytes. Only two commands possible: 12 bytes. Rounding up in case of syntax errors: 16 bytes. Chosen length of the buffer: 256.
- `#define UART_TX_BUF_SIZE 256`
Size allocation for the UART TX software ring buffer. Length of the message in worst case scenario: 59 bytes. Rounded up for little of flexibility with the protocol: 64 bytes. Chosen length of the buffer: 256.
- `#define TX_BUFFER_FULL -1`
Error code returned when the TX circular buffer is completely full.
- `#define RX_BUFFER_EMPTY -1`
Error code returned when attempting to read an empty RX circular buffer.
- `#define INTERRUPT_ATTR __attribute__((__interrupt__))`

Functions

- `int uart1_mikrobus2_setup ()`
Configures UART1 hardware pins, registers, and communication buffers.
- `int uart1_write (const char *msg)`
Non-blocking function to queue a string for UART transmission.
- `int uart1_read_char (void)`
Non-blocking function to fetch a single character from the RX buffer.

- int `uart1_get_tx_count` (void)
Calculates the number of bytes currently waiting in the TX circular buffer.
- int `uart1_get_rx_count` (void)
Calculates the number of unread bytes currently residing in the RX circular buffer.
- void INTERRUPT_ATTR `_U1TXInterrupt` (void)
Hardware interrupt service routine for UART1 transmission.
- void INTERRUPT_ATTR `_U1RXInterrupt` (void)
Hardware interrupt service routine for UART1 reception.

5.23.1 Detailed Description

Header file for the non-blocking UART driver implementation using circular buffers.

5.23.2 Macro Definition Documentation

5.23.2.1 UART_RX_BUF_SIZE

```
#define UART_RX_BUF_SIZE 256
```

Size allocation for the UART RX software ring buffer. Length of single correct command: 6 bytes. Only two commands possible: 12 bytes. Rounding up in case of syntax errors: 16 bytes. Chosen length of the buffer: 256.

•

5.23.2.2 UART_TX_BUF_SIZE

```
#define UART_TX_BUF_SIZE 256
```

Size allocation for the UART TX software ring buffer. Length of the message in worst case scenario: 59 bytes. Rounded up for little of flexibility with the protocol: 64 bytes. Chosen length of the buffer: 256.

•

5.23.3 Function Documentation

5.23.3.1 _U1RXInterrupt()

```
void INTERRUPT_ATTR _U1RXInterrupt (
    void )
```

Hardware interrupt service routine for UART1 reception.

Triggered asynchronously when incoming bytes arrive. It handles hardware overrun errors (OERR) by clearing the bit (which discards corrupted data), and safely copies valid incoming bytes into the RX software circular buffer. If the circular buffer is full, incoming data is silently dropped to prevent memory corruption.

5.23.3.2 _U1TXInterrupt()

```
void INTERRUPT_ATTR _U1TXInterrupt (
    void )
```

Hardware interrupt service routine for UART1 transmission.

Automatically triggered when the UART hardware TX buffer has empty space. It pulls the next available byte from the software circular buffer and pushes it to the hardware register (U1TXREG). If the software buffer empties completely, the interrupt safely disables itself to halt further triggers.

5.23.3.3 uart1_get_rx_count()

```
int uart1_get_rx_count (
    void )
```

Calculates the number of unread bytes currently residing in the RX circular buffer.

Returns

int Number of available bytes waiting to be read.

5.23.3.4 uart1_get_tx_count()

```
int uart1_get_tx_count (  
    void )
```

Calculates the number of bytes currently waiting in the TX circular buffer.

Returns

int Number of pending bytes queued for transmission.

5.23.3.5 uart1_mikrobus2_setup()

```
int uart1_mikrobus2_setup ()
```

Configures UART1 hardware pins, registers, and communication buffers.

Returns

int Always returns 0 upon successful setup.

Sets up the MikroBus2 RX/TX pins, configures the baud rate with High Precision (BRGH=1), and initializes the circular buffer pointers. Rx interrupts are enabled by default, while TX interrupts are kept disabled until data is actively queued.

Returns

int Always returns 0 upon successful setup.

5.23.3.6 uart1_read_char()

```
int uart1_read_char (  
    void )
```

Non-blocking function to fetch a single character from the RX buffer.

Returns

int The received character, or RX_BUFFER_EMPTY if no data is currently available.

Checks the software circular buffer for unread data. If data is present, it returns the byte and advances the read pointer. Because it does not wait for incoming bytes, it is safe to call inside high-frequency processing loops.

Returns

int The received character, or RX_BUFFER_EMPTY if no data is currently available.

5.23.3.7 uart1_write()

```
int uart1_write (  
    const char * msg)
```

Non-blocking function to queue a string for UART transmission.

Parameters

<i>msg</i>	Null-terminated string to transmit.
------------	-------------------------------------

Returns

int Number of bytes successfully queued, or TX_BUFFER_FULL if space is exhausted.

Copies the input string into the TX circular buffer without waiting for hardware transmission to complete. Briefly disables the TX interrupt during buffer manipulation to prevent race conditions, then kickstarts the transmission interrupt if the hardware is idle.

Parameters

<i>msg</i>	Null-terminated string to transmit.
------------	-------------------------------------

Returns

int Number of bytes successfully queued, or TX_BUFFER_FULL if space is exhausted.

5.24 uart.h

[Go to the documentation of this file.](#)

```

00001
00005
00006 #ifndef UART_H
00007 #define UART_H
00008
00009     #include "config.h"
00010
00011     // #define FCY 72000000UL
00012     // #define BAUDRATE 9600 // 115200
00013     // #define BRGVAL ((FCY/BAUDRATE)/16)-1 // only for slow speed mode
00014
00019     #define UART_RX_BUF_SIZE 256
00020
00025     #define UART_TX_BUF_SIZE 256
00026
00028     #define TX_BUFFER_FULL -1
00029
00031     #define RX_BUFFER_EMPTY -1
00032
00037     int uart1_mikrobus2_setup();
00038
00044     int uart1_write(const char *msg);
00045
00050     int uart1_read_char(void);
00051
00056     int uart1_get_tx_count(void);
00057
00062     int uart1_get_rx_count(void);
00063
00064     // int uart1_read_string(char *dest_buffer, int max_len);
00065
00066     /* * Doxygen workaround trick: Hide the __attribute__ macro from the generator
00067      * to prevent naming collisions between the two interrupt functions.
00068      */
00069     #ifdef __DOXYGEN__
00070         #define INTERRUPT_ATTR
00071     #else
00072         #define INTERRUPT_ATTR __attribute__((__interrupt__))
00073     #endif
00074
00078     void INTERRUPT_ATTR _U1TXInterrupt(void);
00079
00083     void INTERRUPT_ATTR _U1RXInterrupt(void);
00084
00085 #endif // UART_H

```

5.25 src/core/fsm.c File Reference

Finite State Machine engine implementation for the robot buggy platform.

```
#include "fsm.h"
```

Typedefs

- typedef [State_t](#)(* **EventHandler**) ([RobotState_t](#) *rover, void *payload)
Function pointer type for FSM event handlers.

Functions

- [State_t](#) **handle_halt** ([RobotState_t](#) *rover, void *payload)
Handler to stop the robot.

- [State_t handle_move](#) ([RobotState_t](#) *rover, void *payload)
Handler to move the robot.
- [State_t handle_obstacle](#) ([RobotState_t](#) *rover, void *payload)
Handler for obstacle detection, initiating avoidance maneuvers.
- [State_t evaluate_obstacle](#) ([RobotState_t](#) *rover, void *payload)
Evaluates an obstacle to determine if avoidance should continue or complete.
- [State_t get_current_state](#) ()
Retrieves the current state of the FSM.
- void [fsm_init](#) ([RobotState_t](#) *rover)
Initializes the FSM and resets mission variables.
- void [dispatch_event](#) ([RobotState_t](#) *rover, [Event_t](#) event, void *payload)
Dispatches an event to the FSM to trigger potential state transitions.

Variables

- const [EventHandler](#) [state_table](#) [[MAX_STATES](#)][[MAX_EVENTS](#)]
State transition table mapping current state and event to an event handler.

5.25.1 Detailed Description

Finite State Machine engine implementation for the robot buggy platform.

Author

Simone Canella, Samuele Isaia, Riccardo Parotelli

Version

1.0.0

5.25.2 Function Documentation

5.25.2.1 dispatch_event()

```
void dispatch_event (
    RobotState\_t * rover,
    Event\_t event,
    void * payload)
```

Dispatches an event to the FSM to trigger potential state transitions.

Submits a system event to the FSM for evaluation and potential state transition.

Parameters

<i>rover</i>	Pointer to the robot state structure.
<i>event</i>	The event being dispatched.
<i>payload</i>	Optional payload associated with the event.

5.25.2.2 evaluate_obstacle()

```
State\_t evaluate_obstacle (
    RobotState\_t * rover,
    void * payload)
```

Evaluates an obstacle to determine if avoidance should continue or complete.

Parameters

<i>rover</i>	Pointer to the robot state structure.
<i>payload</i>	Pointer to the IR distance reading.

Returns

The new state based on evaluation.

5.25.2.3 fsm_init()

```
void fsm_init (
    RobotState_t * rover)
```

Initializes the FSM and resets mission variables.

Initializes the FSM engine, placing it into the default halted state.

Parameters

<i>rover</i>	Pointer to the robot state structure.
--------------	---------------------------------------

5.25.2.4 get_current_state()

```
State_t get_current_state (
    void )
```

Retrieves the current state of the FSM.

Retrieves the current operating state of the finite state machine.

Returns

The current state.

5.25.2.5 handle_halt()

```
State_t handle_halt (
    RobotState_t * rover,
    void * payload)
```

Handler to stop the robot.

Parameters

<i>rover</i>	Pointer to the robot state structure.
<i>payload</i>	Optional event payload.

Returns

The new state (STATE_HALTED).

5.25.2.6 handle_move()

```
State_t handle_move (
    RobotState_t * rover,
    void * payload)
```

Handler to move the robot.

Parameters

<i>rover</i>	Pointer to the robot state structure.
<i>payload</i>	Optional event payload.

Returns

The new state (STATE_MOVING).

5.25.2.7 handle_obstacle()

```
State_t handle_obstacle (
    RobotState_t * rover,
    void * payload)
```

Handler for obstacle detection, initiating avoidance maneuvers.

Parameters

<i>rover</i>	Pointer to the robot state structure.
<i>payload</i>	Optional event payload.

Returns

The new state (STATE_OBSTACLE_AVOIDANCE).

5.25.3 Variable Documentation**5.25.3.1 state_table**

```
const EventHandler state_table[MAX_STATES][MAX_EVENTS]
```

Initial value:

```
= {
    {handle_move,      NULL,      NULL},
    {handle_halt,      handle_obstacle, NULL},
    {handle_halt,      handle_halt,  evaluate_obstacle}
}
```

State transition table mapping current state and event to an event handler.

5.26 src/core/main.c File Reference

Application entry point and main scheduler loop for the buggy.

```
#include "config.h"
#include "robot_state.h"
#include "fsm.h"
#include "tasks.h"
#include "scheduler.h"
```

Functions

- int **main** (void)
Main execution function.

Variables

- int **deadlineMissed**
Counter tracking the number of missed scheduler deadlines.

5.26.1 Detailed Description

Application entry point and main scheduler loop for the buggy.

Author

Simone Canella, Samuele Isaia, Riccardo Parotelli

Version

1.0.0

5.26.2 Function Documentation

5.26.2.1 main()

```
int main (
    void )
```

Main execution function.

Returns

Application exit code.

5.27 src/core/scheduler.c File Reference

Cooperative cyclic task scheduler implementation for the robot buggy platform.

```
#include "config.h"
#include "scheduler.h"
```

Data Structures

- struct [tasky](#)
Internal structure to represent a scheduled task.

Macros

- #define **MAX_TASKS** 16

Functions

- void [scheduler](#) (void)
Core scheduler routine that evaluates and executes ready tasks.
- void [register_task](#) (int initial_offset, int period_ticks, void(*f)(void *), void *arg)
Registers a new task in the scheduler queue.
- void [enable_task](#) (int task_ID)
Enables a registered task.
- void [disable_task](#) (int task_ID)
Disables a registered task.

5.27.1 Detailed Description

Cooperative cyclic task scheduler implementation for the robot buggy platform.

Author

- Simone Canella, Samuele Isaia, Riccardo Parotelli

Version

1.0.0

5.27.2 Function Documentation

5.27.2.1 disable_task()

```
void disable_task (
    int task_ID)
```

Disables a registered task.

Disables a registered task to temporarily halt its execution loop.

Parameters

<i>task_ID</i>	The index of the task to disable.
----------------	-----------------------------------

5.27.2.2 enable_task()

```
void enable_task (
    int task_ID)
```

Enables a registered task.

Enables a registered task to resume its periodic execution loops.

Parameters

<i>task_ID</i>	The index of the task to enable.
----------------	----------------------------------

5.27.2.3 register_task()

```
void register_task (
    int initial_offset,
    int period_ticks,
    void(* f )(void *),
    void * arg)
```

Registers a new task in the scheduler queue.

Registers a new periodic task into the background execution matrix.

Parameters

<i>initial_offset</i>	The initial offset in ticks before first execution.
<i>period_ticks</i>	The recurrence period in ticks.
<i>f</i>	Function pointer to the task routine.
<i>arg</i>	Pointer to pass to the task routine as an argument.

5.27.2.4 scheduler()

```
void scheduler (
    void )
```

Core scheduler routine that evaluates and executes ready tasks.

Main task evaluation and execution loop handler.

5.28 src/core/tasks.c File Reference

Subsystem task implementations and peripheral hardware drivers for the buggy.

```
#include "config.h"
#include "robot_state.h"
#include "fsm.h"
#include "timer.h"
#include "uart.h"
#include "button.h"
#include "spi.h"
#include "BOSCH_BMX055.h"
#include "pwm.h"
#include "adc.h"
#include "scheduler.h"
```

Macros

- #define **MAX_YAW** 80
- #define **MAX_SPEED** 60
- #define **DISTANCE_THRESHOLD** 35.0f
- #define **ROTATION_THRESHOLD** 80.0f
- #define **inputERR** "\$ERR,1*\n"
- #define **PCREF_CMD_MAX_LEN** 24

Functions

- int **car_stopped** ([RobotState_t](#) *rover)
Evaluates if the car is stationary using accelerometer data.
- void **hw_init** ([RobotState_t](#) *rover)
Initializes all necessary hardware peripherals and sets up initial robot state.
- void **read_IR** (void *arg)
Task routine to read IR sensors and battery voltage via ADC.
- void **move** (void *arg)
Task routine to handle movement and obstacle avoidance state logic.
- void **button_polling** (void *arg)
Task routine to poll button states and dispatch related events or UART info.
- void **read_acc_mag** (void *arg)
Task routine to read and process raw accelerometer and magnetometer data.
- void **read_gyro** (void *arg)
Task routine to read gyroscope data and integrate relative yaw.
- void **UART_log_bt** (void *arg)
Task routine to log general system and sensor states over UART via Bluetooth.
- void **UART_log** (void *arg)
Task routine to transmit distance and orientation angles via UART.
- void **UART_get_twist** (void *arg)
Task routine to receive and parse twist commands (\$PCREF) over UART.
- void **blink_A0** (void *arg)
Task routine to toggle the A0 status LED.
- void **UART_bat_log** (void *arg)
Task routine to log the current battery voltage over UART.
- void **blink_lights** (void *arg)
Task routine to manage blinker lights depending on the robot's state.
- void **tasks_init** (void *param)
Initializes and registers all scheduler tasks.

Variables

- int **charReceived** = 0
Flag indicating if a character has been received via UART.

5.28.1 Detailed Description

Subsystem task implementations and peripheral hardware drivers for the buggy.

Author

Simone Canella, Samuele Isaia, Riccardo Parotelli

Version

1.0.0

5.28.2 Function Documentation

5.28.2.1 blink_A0()

```
void blink_A0 (  
    void * arg)
```

Task routine to toggle the A0 status LED.

Parameters

<i>arg</i>	Unused argument.
------------	------------------

5.28.2.2 blinkLights()

```
void blinkLights (  
    void * arg)
```

Task routine to manage blinker lights depending on the robot's state.

Parameters

<i>arg</i>	Unused argument.
------------	------------------

5.28.2.3 button_polling()

```
void button_polling (  
    void * arg)
```

Task routine to poll button states and dispatch related events or UART info.

Parameters

<i>arg</i>	Pointer to the robot state structure.
------------	---------------------------------------

5.28.2.4 car_stopped()

```
int car_stopped (  
    RobotState_t * rover)
```

Evaluates if the car is stationary using accelerometer data.

Parameters

<i>rover</i>	Pointer to the robot state structure.
--------------	---------------------------------------

Returns

1 if stopped, 0 otherwise.

5.28.2.5 hw_init()

```
void hw_init (  
    RobotState_t * rover)
```

Initializes all necessary hardware peripherals and sets up initial robot state.
Configures low-level microcontroller hardware registers and peripherals.

Parameters

<i>rover</i>	Pointer to the robot state structure to configure.
--------------	--

5.28.2.6 move()

```
void move (  
    void * arg)
```

Task routine to handle movement and obstacle avoidance state logic.

Parameters

<i>arg</i>	Pointer to the robot state structure.
------------	---------------------------------------

5.28.2.7 read_acc_mag()

```
void read_acc_mag (  
    void * arg)
```

Task routine to read and process raw accelerometer and magnetometer data.

Parameters

<i>arg</i>	Pointer to the robot state structure.
------------	---------------------------------------

5.28.2.8 read_gyro()

```
void read_gyro (  
    void * arg)
```

Task routine to read gyroscope data and integrate relative yaw.

Parameters

<i>arg</i>	Pointer to the robot state structure.
------------	---------------------------------------

5.28.2.9 read_IR()

```
void read_IR (  
    void * arg)
```

Task routine to read IR sensors and battery voltage via ADC.

Parameters

<i>arg</i>	Pointer to the robot state structure.
------------	---------------------------------------

5.28.2.10 tasks_init()

```
void tasks_init (
    void * param)
```

Initializes and registers all scheduler tasks.

Registers all system runtime operational tasks into the scheduler matrix.

Parameters

<i>param</i>	Pointer to the robot state structure to be passed into tasks.
--------------	---

5.28.2.11 UART_bat_log()

```
void UART_bat_log (
    void * arg)
```

Task routine to log the current battery voltage over UART.

Parameters

<i>arg</i>	Pointer to the robot state structure.
------------	---------------------------------------

5.28.2.12 UART_get_twist()

```
void UART_get_twist (
    void * arg)
```

Task routine to receive and parse twist commands (\$PCREF) over UART.

Parameters

<i>arg</i>	Pointer to the robot state structure.
------------	---------------------------------------

5.28.2.13 UART_log()

```
void UART_log (
    void * arg)
```

Task routine to transmit distance and orientation angles via UART.

Parameters

<i>arg</i>	Pointer to the robot state structure.
------------	---------------------------------------

5.28.2.14 UART_log_bt()

```
void UART_log_bt (
```

```
void * arg)
```

Task routine to log general system and sensor states over UART via Bluetooth.

Parameters

<i>arg</i>	Pointer to the robot state structure.
------------	---------------------------------------

5.29 src/drivers/adc.c File Reference

Analog-to-Digital Converter (ADC) configuration and sampling routines.

```
#include "adc.h"
```

Functions

- void [adc_init](#) (uint8_t channel)
Initializes the ADC hardware and configures target analog pins.
- double [adc_scan_auto](#) (uint8_t channel)
Executes a sample request with automatic hardware conversion.
- void [adc_converter](#) (adcState *adc)
Core processing task to sample sensors and compute physical telemetry metrics.

5.29.1 Detailed Description

Analog-to-Digital Converter (ADC) configuration and sampling routines.

5.29.2 Function Documentation

5.29.2.1 [adc_converter\(\)](#)

```
void adc_converter (
    adcState * adc)
```

Core processing task to sample sensors and compute physical telemetry metrics.

Parameters

<i>adc</i>	Pointer to the adcState structure to populate with metrics.
------------	---

5.29.2.2 [adc_init\(\)](#)

```
void adc_init (
    uint8_t channel)
```

Initializes the ADC hardware and configures target analog pins.

Parameters

<i>channel</i>	The analog channel to initialize.
----------------	-----------------------------------

5.29.2.3 [adc_scan_auto\(\)](#)

```
double adc_scan_auto (
    uint8_t channel)
```

Executes a sample request with automatic hardware conversion.

Parameters

<i>channel</i>	The analog channel to scan.
----------------	-----------------------------

Returns

double The raw ADC conversion result.

5.30 src/drivers/BOSCH_BMX055.c File Reference

SPI driver implementation for the Bosch BMX055 IMU.

```
#include "BOSCH_BMX055.h"
#include <xc.h>
#include "timer.h"
#include "spi.h"
```

Functions

- void [BMX055_setup](#) ()
Initializes the BMX055 sensor and configures the SPI bus parameters.
- void [BMX055_mag_wakeup](#) ()
Executes the wake-up sequence for the BMX055 magnetometer.
- int [format12bit](#) ()
Reads two bytes via SPI and formats them as a 12-bit signed integer.
- int [format13bit](#) ()
Reads two bytes via SPI and formats them as a 13-bit signed integer.
- int [format15bit](#) ()
Reads two bytes via SPI and formats them as a 15-bit signed integer.
- int [format16bit](#) ()
Reads two bytes via SPI and formats them as a standard 16-bit signed integer.
- [IMUdata imuRead](#) (int sensor)
Reads a full frame of multi-axis data from the specified IMU sensor.

Variables

- volatile unsigned int **DATA_LSB**
Global volatile buffers for temporarily storing raw SPI byte transactions.
- volatile unsigned int **DATA_MSB**

5.30.1 Detailed Description

SPI driver implementation for the Bosch BMX055 IMU.

This file contains the initialization sequences, power management routines, and data extraction functions required to interface with the Accelerometer, Gyroscope, and Magnetometer components of the BMX055 sensor over SPI.

5.30.2 Function Documentation

5.30.2.1 BMX055_mag_wakeup()

```
void BMX055_mag_wakeup ()
```

Executes the wake-up sequence for the BMX055 magnetometer.

- Forces the magnetometer interface to switch from I2C to SPI by toggling the MAG_CS pin.
- Transitions the magnetometer from Suspend Mode to Sleep Mode by writing 0x01 to register 0x4B.

- Transitions the magnetometer from Sleep Mode to Active Mode by writing 0x00 to register 0x4C.
- Utilizes TIMER1 delays between transmissions to allow for power mode switching times.

5.30.2.2 BMX055_setup()

```
void BMX055_setup ()
```

Initializes the BMX055 sensor and configures the SPI bus parameters.

- Sets Chip Select (CS) lines for the Accelerometer, Gyroscope, and Magnetometer to high (inactive).
- Temporarily disables the SPI module to configure protocol settings matching the BMX055 requirements.
- Configures the SPI clock polarity to idle LOW (CKP = 0) and sets data changes to occur on transition from active to idle (CKE = 1).
- Sets the accelerometer's initial bandwidth to 1000 Hz by writing value 15 to register 0x10.

5.30.2.3 format12bit()

```
int format12bit ()
```

Reads two bytes via SPI and formats them as a 12-bit signed integer.

- Masks the lower 4 bits of the LSB with 0b11110000 to isolate the valid data.
- Shifts the MSB left by 8 bits and combines it with the LSB.
- Divides the combined short by 16 to properly align the 12-bit value.

Returns

int The formatted 12-bit signed data value.

5.30.2.4 format13bit()

```
int format13bit ()
```

Reads two bytes via SPI and formats them as a 13-bit signed integer.

- Masks the lower 3 bits of the LSB with 0b11111000 to isolate the valid data.
- Shifts the MSB left by 8 bits and combines it with the LSB.
- Divides the combined short by 8 to properly align the 13-bit value.

Returns

int The formatted 13-bit signed data value.

5.30.2.5 format15bit()

```
int format15bit ()
```

Reads two bytes via SPI and formats them as a 15-bit signed integer.

- Masks the lowest bit of the LSB with 0b11111110 to isolate the valid data.
- Shifts the MSB left by 8 bits and combines it with the LSB.
- Divides the combined short by 2 to properly align the 15-bit value.

Returns

int The formatted 15-bit signed data value.

5.30.2.6 format16bit()

```
int format16bit ()
```

Reads two bytes via SPI and formats them as a standard 16-bit signed integer.

- Shifts the MSB left by 8 bits and performs a bitwise OR with the LSB.

Returns

int The combined 16-bit signed data value.

5.30.2.7 imuRead()

```
IMUdata imuRead (
    int sensor)
```

Reads a full frame of multi-axis data from the specified IMU sensor.

- Dispatches SPI reads depending on the requested sensor macro ([ACC](#), [GYR](#), [MAG](#)).
- Sets the MSB of the register address to 1 (0x80) to indicate a read operation.
- Accelerometer (ACC): Reads from register 0x02 and formats X, Y, and Z axes as 12-bit integers.
- Gyroscope (GYR): Reads from register 0x02 and formats X, Y, and Z axes as 16-bit integers.
- Magnetometer (MAG): Reads from register 0x42, formats X and Y axes as 13-bit integers, and the Z axis as a 15-bit integer.

Parameters

<i>sensor</i>	The target sensor peripheral to read (ACC , GYR , or MAG).
---------------	--

Returns

[IMUdata](#) A structure containing the extracted x, y, and z axis measurements.

5.31 src/drivers/button.c File Reference

Implementation of button initialization and debouncing logic.

```
#include "button.h"
```

Functions

- void [button_init](#) ([ButtonState](#) *btn)
Initializes a button to its default (unpressed) state.
- int [button_has_just_been_pressed](#) ([ButtonState](#) *btn, int current_pin_state)
Evaluates the button state and handles debouncing.

5.31.1 Detailed Description

Implementation of button initialization and debouncing logic.

5.31.2 Function Documentation

5.31.2.1 button_has_just_been_pressed()

```
int button_has_just_been_pressed (
    ButtonState * btn,
    int current_pin_state)
```

Evaluates the button state and handles debouncing.

- Returns 1 ONLY on the exact cycle the button is validly pressed. Returns 0 otherwise. Decrements the debounce counter if a state change is detected, and updates the state only when the counter reaches 0.

Parameters

<i>btn</i>	Pointer to the active ButtonState structure tracking this pin.
<i>current_pin_state</i>	The current digital reading of the physical button pin (0 or 1).

Returns

int Returns 1 if a valid press is detected, 0 otherwise.

5.31.2.2 button_init()

```
void button_init (
    ButtonState * btn)
```

Initializes a button to its default (unpressed) state.

- Sets the previous state to 1, assuming pull-up resistors are used, and resets the debounce counter to MAXDEBCYCLE.

Parameters

<i>btn</i>	Pointer to the ButtonState structure to be initialized.
------------	---

5.32 src/drivers/pwm.c File Reference

Pulse Width Modulation (PWM) and motor control interface.

```
#include "pwm.h"
```

Functions

- void [pwm_init](#) (void)
Initializes the motor control pins and Output Compare peripherals.
- int [pwm_set_duty_cycle](#) (int percent, unsigned int reg)
Converts a percentage into a hardware register duty cycle value.
- int [clamp](#) (int value)
Clamps an input value to the absolute limits of the motor saturation [-100, 100].
- void [execute_motion](#) (int v, int w)
Computes differential kinematics and updates H-Bridge duty cycles.

5.32.1 Detailed Description

Pulse Width Modulation (PWM) and motor control interface.

This file configures the Output Compare (OC) modules to generate 10 kHz PWM signals for a 4-wheel differential drive robotic platform. It handles the kinematic mixing of linear velocity and angular yaw rate into individual track speeds, applying directional logic to the H-Bridge pins.

5.32.2 Function Documentation

5.32.2.1 clamp()

```
int clamp (
    int value)
```

Clamps an input value to the absolute limits of the motor saturation [-100, 100].

Parameters

<i>value</i>	The raw kinematic speed demand.
--------------	---------------------------------

Returns

int The bounded speed value.

5.32.2.2 execute_motion()

```
void execute_motion (
    int v,
    int w)
```

Computes differential kinematics and updates H-Bridge duty cycles.

Converts a target forward velocity and angular yaw rate into independent left and right track speeds. It applies threshold clamps and routes the resulting magnitudes to the appropriate forward or backward OC module.

Parameters

<i>v</i>	Target linear velocity (-100 to 100).
<i>w</i>	Target angular yaw rate (-100 to 100).

5.32.2.3 pwm_init()

```
void pwm_init (
    void )
```

Initializes the motor control pins and Output Compare peripherals.

Configures the directional pins (RD1-RD4) as outputs and maps them to specific Re-mappable Pins (RP65-RP68). It initializes four Output Compare modules (OC1-OC4) in edge-aligned PWM mode with a base frequency of 10 kHz.

5.32.2.4 pwm_set_duty_cycle()

```
int pwm_set_duty_cycle (
    int percent,
    unsigned int reg)
```

Converts a percentage into a hardware register duty cycle value.

Parameters

<i>percent</i>	The desired duty cycle ranging from 0 to 100.
<i>reg</i>	The period register boundary (OCxRS) to scale against.

Returns

int The calculated hardware integer to be written to OCxR.

5.33 src/drivers/spi.c File Reference

SPI interface driver and hardware configuration.

```
#include "timer.h"
```

Functions

- void [spi1_mikrobus1_setup](#) ()
Configures the SPI1 peripheral pins, control registers, and default chip selects.
- unsigned int [spi1_write](#) (unsigned int data)
Transmits and receives a single byte over the SPI bus.

5.33.1 Detailed Description

SPI interface driver and hardware configuration.

This module provides the hardware setup and primitive transmit/receive functions for the SPI1 peripheral, specifically routed to the MikroBus1 header footprint.

5.33.2 Function Documentation**5.33.2.1 spi1_mikrobus1_setup()**

```
void spil_mikrobus1_setup ()
```

Configures the SPI1 peripheral pins, control registers, and default chip selects.

- Maps the MISO line to input pin RA1 (RP117).
- Maps the MOSI line to output pin RF13 (RP109).
- Maps the SCK (Serial Clock) line to output pin RF12 (RP108).
- Initializes three independent Chip Select (CS) pins (RB3, RB4, RD6) as digital outputs idling HIGH (inactive).
- Configures the SPI core for Master Mode, 8-bit data width, and sets prescalers to control the SCK frequency.
- Enables the SPI1 module.

5.33.2.2 spi1_write()

```
unsigned int spil_write (
    unsigned int data)
```

Transmits and receives a single byte over the SPI bus.

This is a blocking, synchronous hardware call. SPI communication is full-duplex, meaning a byte is always received simultaneously as a byte is transmitted.

Parameters

<i>data</i>	The 8-bit data payload to transmit.
-------------	-------------------------------------

Returns

unsigned int The 8-bit data payload received from the slave device.

5.34 src/drivers/timer.c File Reference

Hardware timer configuration and delay management routines.

```
#include "timer.h"
```

Functions

- int [tmr_setup_period](#) (int tmr, float ms)
Configures and starts a specific timer for a given period.
- int [tmr_wait_period](#) (int tmr)
Performs a blocking wait until the specified timer period expires.
- int [tmr_wait_ms](#) (int tmr, float ms)
Executes a synchronous, one-shot blocking delay.

5.34.1 Detailed Description

Hardware timer configuration and delay management routines.

- This file implements the setup and control functions for hardware timers TIMER1 through TIMER5. It calculates prescaler and period register values dynamically based on a 72 MHz system instruction frequency (Fcy) to provide accurate timing and blocking delays.

5.34.2 Function Documentation**5.34.2.1 tmr_setup_period()**

```
int tmr_setup_period (
    int tmr,
    float ms)
```

Configures and starts a specific timer for a given period.

- Dynamically selects the optimal prescaler (1:8, 1:64, or 1:256) based on the target milliseconds to avoid register overflow. It resets the timer, clears its interrupt flag, configures the period register (PRx), and starts it.

•

Parameters

<i>tmr</i>	The hardware timer to configure (e.g., TIMER1, TIMER2, ..., TIMER5).
<i>ms</i>	The desired timer period in milliseconds. Max supported period is ~232 ms.

Returns

int Returns 0 on successful configuration, or NOTMR if an invalid timer is passed.

5.34.2.2 tmr_wait_ms()

```
int tmr_wait_ms (
    int tmr,
    float ms)
```

Executes a synchronous, one-shot blocking delay.

- A convenience wrapper that automatically sets up the timer, waits for the requested period to expire, and cleanly shuts down the timer afterward to prevent continuous background counting.

- **Parameters**

<i>tmr</i>	The hardware timer to use for the delay (e.g., TIMER1, TIMER2, ..., TIMER5).
<i>ms</i>	The duration to block execution, in milliseconds.

Returns

int Returns 0 upon completion, or NOTMR if an invalid timer is passed.

5.34.2.3 `tmr_wait_period()`

```
int tmr_wait_period (
    int tmr)
```

Performs a blocking wait until the specified timer period expires.

- Polls the hardware interrupt flag for the given timer. If the flag is already set when this function is called, it detects a missed deadline. Otherwise, it holds execution in a while-loop until the hardware timer trips the flag.

•

Parameters

<i>tmr</i>	The hardware timer to monitor (e.g., TIMER1, TIMER2, ..., TIMER5).
------------	--

Returns

int Returns 0 upon successful wait completion, TMREXP if the deadline was missed before polling began, or NOTMR for an invalid timer.

5.35 `src/drivers/uart.c` File Reference

Non-blocking UART driver implementation using circular buffers and interrupts.

```
#include "uart.h"
#include "config.h"
#include "timer.h"
```

Data Structures

- struct [circular_buffer](#)
Internal circular buffer structure for non-blocking data queuing.

Functions

- int [uart1_mikrobus2_setup](#) ()
Configures UART1 hardware pins, registers, and communication buffers.
- int [uart1_write](#) (const char *msg)
Non-blocking function to queue a string for UART transmission.
- void [_U1TXInterrupt](#) (void)
Hardware interrupt service routine for UART1 transmission.
- void [_U1RXInterrupt](#) (void)
Hardware interrupt service routine for UART1 reception.
- int [uart1_read_char](#) (void)
Non-blocking function to fetch a single character from the RX buffer.
- int [uart1_get_tx_count](#) (void)
Calculates the number of bytes currently waiting in the TX circular buffer.
- int [uart1_get_rx_count](#) (void)
Calculates the number of unread bytes currently residing in the RX circular buffer.

Variables

- volatile struct [circular_buffer](#) **U1RX_buffer**
Global instances of the TX and RX circular buffers.
- volatile struct [circular_buffer](#) **U1TX_buffer**
- volatile char **uart_rx_data_buff** [[UART_RX_BUF_SIZE](#)]
Underlying memory arrays allocated for the circular buffers.
- volatile char **uart_tx_data_buff** [[UART_TX_BUF_SIZE](#)]

5.35.1 Detailed Description

Non-blocking UART driver implementation using circular buffers and interrupts.

This module provides a fully non-blocking serial communication interface. By utilizing interrupt service routines (ISRs) and software circular buffers for both transmission and reception, the main execution thread is never halted waiting for hardware shift registers to clear.

5.35.2 Function Documentation

5.35.2.1 _U1RXInterrupt()

```
void _U1RXInterrupt (  
    void )
```

Hardware interrupt service routine for UART1 reception.

Triggered asynchronously when incoming bytes arrive. It handles hardware overrun errors (OERR) by clearing the bit (which discards corrupted data), and safely copies valid incoming bytes into the RX software circular buffer. If the circular buffer is full, incoming data is silently dropped to prevent memory corruption.

5.35.2.2 _U1TXInterrupt()

```
void _U1TXInterrupt (  
    void )
```

Hardware interrupt service routine for UART1 transmission.

Automatically triggered when the UART hardware TX buffer has empty space. It pulls the next available byte from the software circular buffer and pushes it to the hardware register (U1TXREG). If the software buffer empties completely, the interrupt safely disables itself to halt further triggers.

5.35.2.3 uart1_get_rx_count()

```
int uart1_get_rx_count (  
    void )
```

Calculates the number of unread bytes currently residing in the RX circular buffer.

Returns

int Number of available bytes waiting to be read.

5.35.2.4 uart1_get_tx_count()

```
int uart1_get_tx_count (  
    void )
```

Calculates the number of bytes currently waiting in the TX circular buffer.

Returns

int Number of pending bytes queued for transmission.

5.35.2.5 uart1_mikrobus2_setup()

```
int uart1_mikrobus2_setup ()
```

Configures UART1 hardware pins, registers, and communication buffers.

Sets up the MikroBus2 RX/TX pins, configures the baud rate with High Precision (BRGH=1), and initializes the circular buffer pointers. Rx interrupts are enabled by default, while TX interrupts are kept disabled until data is actively queued.

Returns

int Always returns 0 upon successful setup.

5.35.2.6 uart1_read_char()

```
int uart1_read_char (  
    void )
```

Non-blocking function to fetch a single character from the RX buffer.

Checks the software circular buffer for unread data. If data is present, it returns the byte and advances the read pointer. Because it does not wait for incoming bytes, it is safe to call inside high-frequency processing loops.

Returns

int The received character, or RX_BUFFER_EMPTY if no data is currently available.

5.35.2.7 uart1_write()

```
int uart1_write (  
    const char * msg)
```

Non-blocking function to queue a string for UART transmission.

Copies the input string into the TX circular buffer without waiting for hardware transmission to complete. Briefly disables the TX interrupt during buffer manipulation to prevent race conditions, then kickstarts the transmission interrupt if the hardware is idle.

Parameters

<i>msg</i>	Null-terminated string to transmit.
------------	-------------------------------------

Returns

int Number of bytes successfully queued, or TX_BUFFER_FULL if space is exhausted.

Index

- `_U1RXInterrupt`
 - `uart.c`, [71](#)
 - `uart.h`, [47](#)
 - `_U1TXInterrupt`
 - `uart.c`, [71](#)
 - `uart.h`, [47](#)
- `ACC_CHIP_ID`
 - `config.h`, [18](#)
- `ACC_CS_LAT`
 - `config.h`, [18](#)
- `ACC_CS_TRIS`
 - `config.h`, [18](#)
- `acc_raw`
 - `SystemSensors_t`, [14](#)
- `acc_x`
 - `SystemSensors_t`, [14](#)
- `acc_y`
 - `SystemSensors_t`, [15](#)
- `acc_z`
 - `SystemSensors_t`, [15](#)
- `adc.c`
 - `adc_converter`, [61](#)
 - `adc_init`, [61](#)
 - `adc_scan_auto`, [61](#)
- `adc.h`
 - `adc_converter`, [34](#)
 - `adc_init`, [34](#)
 - `adc_scan_auto`, [34](#)
- `adc_converter`
 - `adc.c`, [61](#)
 - `adc.h`, [34](#)
- `adc_init`
 - `adc.c`, [61](#)
 - `adc.h`, [34](#)
- `adc_scan_auto`
 - `adc.c`, [61](#)
 - `adc.h`, [34](#)
- `adc_status`
 - `SystemSensors_t`, [15](#)
- `adcState`, [9](#)
 - `battery_voltage`, [9](#)
 - `ir_distance`, [9](#)
- `arg`
 - `tasky`, [16](#)
- `attitude`
 - `RobotState_t`, [12](#)
- `battery_voltage`
 - `adcState`, [9](#)
- `SystemSensors_t`, [15](#)
- `BAUDRATE`
 - `config.h`, [18](#)
- `blink_A0`
 - `tasks.c`, [57](#)
- `blink_lights`
 - `tasks.c`, [57](#)
- `BMX055_mag_wakeup`
 - `BOSCH_BMX055.c`, [62](#)
 - `BOSCH_BMX055.h`, [36](#)
- `BMX055_setup`
 - `BOSCH_BMX055.c`, [63](#)
 - `BOSCH_BMX055.h`, [36](#)
- `BOSCH_BMX055.c`
 - `BMX055_mag_wakeup`, [62](#)
 - `BMX055_setup`, [63](#)
 - `format12bit`, [63](#)
 - `format13bit`, [63](#)
 - `format15bit`, [63](#)
 - `format16bit`, [63](#)
 - `imuRead`, [64](#)
- `BOSCH_BMX055.h`
 - `BMX055_mag_wakeup`, [36](#)
 - `BMX055_setup`, [36](#)
 - `imuRead`, [36](#)
- `BRGVAL`
 - `config.h`, [18](#)
- `btn_re8`
 - `SystemIO_t`, [14](#)
- `btn_re9`
 - `SystemIO_t`, [14](#)
- `BTN_T1_PORT`
 - `config.h`, [18](#)
- `BTN_T1_TRIS`
 - `config.h`, [19](#)
- `BTN_T2_PORT`
 - `config.h`, [19](#)
- `BTN_T2_TRIS`
 - `config.h`, [19](#)
- `button.c`
 - `button_has_just_been_pressed`, [65](#)
 - `button_init`, [65](#)
- `button.h`
 - `button_has_just_been_pressed`, [38](#)
 - `button_init`, [38](#)
 - `MAXDEBCYCLE`, [37](#)
- `button_has_just_been_pressed`
 - `button.c`, [65](#)
 - `button.h`, [38](#)

- button_init
 - button.c, [65](#)
 - button.h, [38](#)
- button_polling
 - tasks.c, [57](#)
- ButtonState, [9](#)
 - debounce_counter, [10](#)
 - prev_state, [10](#)
- car_stopped
 - tasks.c, [57](#)
- char_received
 - SystemIO_t, [14](#)
- circular_buffer, [10](#)
 - data, [10](#)
 - idxH, [10](#)
 - idxT, [10](#)
 - size, [10](#)
- clamp
 - pwm.c, [66](#)
- cmd_vel
 - RobotState_t, [12](#)
- config.h
 - ACC_CHIP_ID, [18](#)
 - ACC_CS_LAT, [18](#)
 - ACC_CS_TRIS, [18](#)
 - BAUDRATE, [18](#)
 - BRGVAL, [18](#)
 - BTN_T1_PORT, [18](#)
 - BTN_T1_TRIS, [19](#)
 - BTN_T2_PORT, [19](#)
 - BTN_T2_TRIS, [19](#)
 - FCY, [19](#)
 - GYR_CHIP_ID, [19](#)
 - GYR_CS_LAT, [19](#)
 - GYR_CS_TRIS, [19](#)
 - LED1_LAT, [19](#)
 - LED1_TRIS, [19](#)
 - LIL_LAT, [19](#)
 - LIL_TRIS, [19](#)
 - LSL_LAT, [19](#)
 - LSL_TRIS, [20](#)
 - MAG_CHIP_ID, [20](#)
 - MAG_CS_LAT, [20](#)
 - MAG_CS_TRIS, [20](#)
 - MAG_OFFSET_X, [20](#)
 - MAG_OFFSET_Y, [20](#)
 - MAG_OFFSET_Z, [20](#)
 - MSG_ERR_INPUT, [20](#)
 - RAD_TO_DEG, [20](#)
 - RSL_LAT, [20](#)
 - RSL_TRIS, [20](#)
 - SPI1_MISO_RPIN, [20](#)
 - SPI1_MOSI_RPIN, [21](#)
 - SPI1_PPRES, [21](#)
 - SPI1_SCK_RPIN, [21](#)
 - SPI1_SPRES, [21](#)
 - UART1_RX_RPIN, [21](#)
 - UART1_TX_RPIN, [21](#)
- current_phase
 - MissionControl_t, [11](#)
- data
 - circular_buffer, [10](#)
- debounce_counter
 - ButtonState, [10](#)
- disable_task
 - scheduler.c, [55](#)
 - scheduler.h, [28](#)
- dispatch_event
 - fsm.c, [51](#)
 - fsm.h, [23](#)
- enable
 - tasky, [16](#)
- enable_task
 - scheduler.c, [55](#)
 - scheduler.h, [28](#)
- evaluate_obstacle
 - fsm.c, [51](#)
- EVENT_MANEUVER_COMPLETE
 - fsm.h, [23](#)
- EVENT_OBSTACLE_DETECTED
 - fsm.h, [23](#)
- EVENT_RE8_PRESSED
 - fsm.h, [23](#)
- Event_t
 - fsm.h, [23](#)
- execute_motion
 - pwm.c, [66](#)
 - pwm.h, [39](#)
- f
 - tasky, [16](#)
- FCY
 - config.h, [19](#)
- format12bit
 - BOSCH_BMX055.c, [63](#)
- format13bit
 - BOSCH_BMX055.c, [63](#)
- format15bit
 - BOSCH_BMX055.c, [63](#)
- format16bit
 - BOSCH_BMX055.c, [63](#)
- fsm.c
 - dispatch_event, [51](#)
 - evaluate_obstacle, [51](#)
 - fsm_init, [52](#)
 - get_current_state, [52](#)
 - handle_halt, [52](#)
 - handle_move, [52](#)
 - handle_obstacle, [53](#)
 - state_table, [53](#)
- fsm.h
 - dispatch_event, [23](#)
 - EVENT_MANEUVER_COMPLETE, [23](#)
 - EVENT_OBSTACLE_DETECTED, [23](#)
 - EVENT_RE8_PRESSED, [23](#)

- Event_t, 23
- fsm_init, 24
- get_current_state, 24
- MAX_EVENTS, 23
- MAX_STATES, 23
- STATE_HALTED, 23
- STATE_MOVING, 23
- STATE_OBSTACLE_AVOIDANCE, 23
- State_t, 23
- fsm_init
 - fsm.c, 52
 - fsm.h, 24
- get_current_state
 - fsm.c, 52
 - fsm.h, 24
- GYR_CHIP_ID
 - config.h, 19
- GYR_CS_LAT
 - config.h, 19
- GYR_CS_TRIS
 - config.h, 19
- gyr_raw
 - SystemSensors_t, 15
- handle_halt
 - fsm.c, 52
- handle_move
 - fsm.c, 52
- handle_obstacle
 - fsm.c, 53
- hw_init
 - tasks.c, 59
 - tasks.h, 31
- idxH
 - circular_buffer, 10
- idxT
 - circular_buffer, 10
- IMUdata, 10
 - x, 11
 - y, 11
 - z, 11
- imuRead
 - BOSCH_BMX055.c, 64
 - BOSCH_BMX055.h, 36
- include/core/config.h, 17, 21
- include/core/fsm.h, 22, 24
- include/core/robot_state.h, 25, 26
- include/core/scheduler.h, 27, 30
- include/core/tasks.h, 31, 33
- include/drivers/adc.h, 33, 34
- include/drivers/BOSCH_BMX055.h, 35, 37
- include/drivers/button.h, 37, 38
- include/drivers/pwm.h, 39, 40
- include/drivers/spi.h, 40, 41
- include/drivers/timer.h, 42, 46
- include/drivers/uart.h, 46, 50
- io
 - RobotState_t, 12
- ir_distance
 - adcState, 9
 - SystemSensors_t, 15
- LED1_LAT
 - config.h, 19
- LED1_TRIS
 - config.h, 19
- LIL_LAT
 - config.h, 19
- LIL_TRIS
 - config.h, 19
- LSL_LAT
 - config.h, 19
- LSL_TRIS
 - config.h, 20
- MAG_CHIP_ID
 - config.h, 20
- MAG_CS_LAT
 - config.h, 20
- MAG_CS_TRIS
 - config.h, 20
- MAG_OFFSET_X
 - config.h, 20
- MAG_OFFSET_Y
 - config.h, 20
- MAG_OFFSET_Z
 - config.h, 20
- mag_raw
 - SystemSensors_t, 15
- main
 - main.c, 54
- Main Page, 1
- main.c
 - main, 54
- MAX_EVENTS
 - fsm.h, 23
- MAX_STATES
 - fsm.h, 23
- MAXDEBCYCLE
 - button.h, 37
- mission
 - RobotState_t, 12
- MissionControl_t, 11
 - current_phase, 11
 - next_phase, 11
 - obstacle_counter, 11
 - phase_counter, 12
- move
 - tasks.c, 59
- MSG_ERR_INPUT
 - config.h, 20
- N
 - tasky, 16
- n
 - tasky, 16

- next_phase
 - MissionControl_t, 11
- OBAV_PHASE_EXIT
 - robot_state.h, 26
- OBAV_PHASE_MOVE_FW
 - robot_state.h, 26
- OBAV_PHASE_ROTATE_CCW
 - robot_state.h, 26
- OBAV_PHASE_ROTATE_CW
 - robot_state.h, 26
- OBAV_PHASE_STOP
 - robot_state.h, 26
- ObAvPhase_t
 - robot_state.h, 26
- obstacle_counter
 - MissionControl_t, 11
- phase_counter
 - MissionControl_t, 12
- pitch
 - SystemAttitude_t, 13
- prev_state
 - ButtonState, 10
- pwm.c
 - clamp, 66
 - execute_motion, 66
 - pwm_init, 66
 - pwm_set_duty_cycle, 66
- pwm.h
 - execute_motion, 39
 - pwm_init, 39
 - pwm_set_duty_cycle, 40
- pwm_init
 - pwm.c, 66
 - pwm.h, 39
- pwm_set_duty_cycle
 - pwm.c, 66
 - pwm.h, 40
- RAD_TO_DEG
 - config.h, 20
- read_acc_mag
 - tasks.c, 59
- read_gyro
 - tasks.c, 59
- read_IR
 - tasks.c, 59
- register_task
 - scheduler.c, 55
 - scheduler.h, 28
- relative_yaw
 - SystemAttitude_t, 13
- robot_state.h
 - OBAV_PHASE_EXIT, 26
 - OBAV_PHASE_MOVE_FW, 26
 - OBAV_PHASE_ROTATE_CCW, 26
 - OBAV_PHASE_ROTATE_CW, 26
 - OBAV_PHASE_STOP, 26
 - ObAvPhase_t, 26
 - RobotState_t, 12
 - attitude, 12
 - cmd_vel, 12
 - io, 12
 - mission, 12
 - sensors, 12
- roll
 - SystemAttitude_t, 13
- RSL_LAT
 - config.h, 20
- RSL_TRIS
 - config.h, 20
- rx_cmd
 - SystemIO_t, 14
- scheduler
 - scheduler.c, 55
 - scheduler.h, 30
- scheduler.c
 - disable_task, 55
 - enable_task, 55
 - register_task, 55
 - scheduler, 55
- scheduler.h
 - disable_task, 28
 - enable_task, 28
 - register_task, 28
 - scheduler, 30
 - tick_ms, 27
- sensors
 - RobotState_t, 12
- size
 - circular_buffer, 10
- spi.c
 - spi1_mikrobus1_setup, 67
 - spi1_write, 67
- spi.h
 - spi1_mikrobus1_setup, 41
 - spi1_write, 41
- spi1_mikrobus1_setup
 - spi.c, 67
 - spi.h, 41
- SPI1_MISO_RPIN
 - config.h, 20
- SPI1_MOSI_RPIN
 - config.h, 21
- SPI1_PPRE
 - config.h, 21
- SPI1_SCK_RPIN
 - config.h, 21
- SPI1_SPRE
 - config.h, 21
- spi1_write
 - spi.c, 67
 - spi.h, 41
- src/core/fsm.c, 50
- src/core/main.c, 53
- src/core/scheduler.c, 54

- src/core/tasks.c, 55
- src/drivers/adc.c, 61
- src/drivers/BOSCH_BMX055.c, 62
- src/drivers/button.c, 64
- src/drivers/pwm.c, 65
- src/drivers/spi.c, 67
- src/drivers/timer.c, 68
- src/drivers/uart.c, 70
- STATE_HALTED
 - fsm.h, 23
- STATE_MOVING
 - fsm.h, 23
- STATE_OBSTACLE_AVOIDANCE
 - fsm.h, 23
- State_t
 - fsm.h, 23
- state_table
 - fsm.c, 53
- SystemAttitude_t, 13
 - pitch, 13
 - relative_yaw, 13
 - roll, 13
 - yaw, 13
- SystemIO_t, 13
 - btn_re8, 14
 - btn_re9, 14
 - char_received, 14
 - rx_cmd, 14
 - tx_buffer, 14
- SystemSensors_t, 14
 - acc_raw, 14
 - acc_x, 14
 - acc_y, 15
 - acc_z, 15
 - adc_status, 15
 - battery_voltage, 15
 - gyr_raw, 15
 - ir_distance, 15
 - mag_raw, 15
- tasks.c
 - blink_A0, 57
 - blink_lights, 57
 - button_polling, 57
 - car_stopped, 57
 - hw_init, 59
 - move, 59
 - read_acc_mag, 59
 - read_gyro, 59
 - read_IR, 59
 - tasks_init, 60
 - UART_bat_log, 60
 - UART_get_twist, 60
 - UART_log, 60
 - UART_log_bt, 60
- tasks.h
 - hw_init, 31
 - tasks_init, 31
- tasks_init
 - tasks.c, 60
 - tasks.h, 31
- tasky, 15
 - arg, 16
 - enable, 16
 - f, 16
 - N, 16
 - n, 16
- tick_ms
 - scheduler.h, 27
- timer.c
 - tmr_setup_period, 68
 - tmr_wait_ms, 68
 - tmr_wait_period, 70
- timer.h
 - tmr_setup_period, 42
 - tmr_wait_ms, 44
 - tmr_wait_period, 44
- tmr_setup_period
 - timer.c, 68
 - timer.h, 42
- tmr_wait_ms
 - timer.c, 68
 - timer.h, 44
- tmr_wait_period
 - timer.c, 70
 - timer.h, 44
- Twist_t, 16
 - v, 16
 - w, 16
- tx_buffer
 - SystemIO_t, 14
- uart.c
 - _U1RXInterrupt, 71
 - _U1TXInterrupt, 71
 - uart1_get_rx_count, 71
 - uart1_get_tx_count, 71
 - uart1_mikrobus2_setup, 71
 - uart1_read_char, 72
 - uart1_write, 72
- uart.h
 - _U1RXInterrupt, 47
 - _U1TXInterrupt, 47
 - uart1_get_rx_count, 47
 - uart1_get_tx_count, 48
 - uart1_mikrobus2_setup, 48
 - uart1_read_char, 48
 - uart1_write, 48
 - UART_RX_BUF_SIZE, 47
 - UART_TX_BUF_SIZE, 47
- uart1_get_rx_count
 - uart.c, 71
 - uart.h, 47
- uart1_get_tx_count
 - uart.c, 71
 - uart.h, 48
- uart1_mikrobus2_setup
 - uart.c, 71

- uart.h, [48](#)
- uart1_read_char
 - uart.c, [72](#)
 - uart.h, [48](#)
- UART1_RX_RPIN
 - config.h, [21](#)
- UART1_TX_RPIN
 - config.h, [21](#)
- uart1_write
 - uart.c, [72](#)
 - uart.h, [48](#)
- UART_bat_log
 - tasks.c, [60](#)
- UART_get_twist
 - tasks.c, [60](#)
- UART_log
 - tasks.c, [60](#)
- UART_log_bt
 - tasks.c, [60](#)
- UART_RX_BUF_SIZE
 - uart.h, [47](#)
- UART_TX_BUF_SIZE
 - uart.h, [47](#)
- v
 - Twist_t, [16](#)
- w
 - Twist_t, [16](#)
- x
 - IMUdata, [11](#)
- y
 - IMUdata, [11](#)
- yaw
 - SystemAttitude_t, [13](#)
- z
 - IMUdata, [11](#)